UNIVERSIDADE FEDERAL DE SANTA CATARINA
CENTRO TECNOLÓGICO
DEPARTAMENTO DE INFORMÁTICA E ESTATÍSTICA
CIÊNCIA DA COMPUTAÇÃO

Davi Menegaz Junkes

Gateway Semântico para Tradução de Mensagens ROS 2 para o Formato SmartData

Florianópolis
2026

Davi Menegaz Junkes

Gateway Semântico para Tradução de Mensagens ROS 2 para o Formato SmartData

Trabalho de Conclusão de Curso submetido ao Curso de Graduação em Ciência da Computação do Centro Tecnológico da Universidade Federal de Santa Catarina como requisito para obtenção do título de Bacharel em Ciência da Computação.

Orientador: Prof. Antônio Augusto Fröhlich, Phd.
Coorientador: Matheus Wagner, Me.
Coorientador: José Luis Conradi Hoffmann, Phd.

Florianópolis
2026

Davi Menegaz Junkes

Gateway Semântico para Tradução de Mensagens ROS 2 para o Formato SmartData

Este Trabalho de Conclusão de Curso foi julgado adequado para obtenção do Título de Bacharel em Ciência da Computação e aprovado em sua forma final pelo curso de Graduação em Ciência da Computação.

Florianópolis, _____ de _____ de 2026.

Banca Examinadora:

Prof. Antônio Augusto Fröhlich, Phd.
Orientador
Universidade Federal de Santa Catarina

Matheus Wagner, Me.
Coorientador
Universidade Federal de Santa Catarina

José Luis Conradi Hoffmann, Phd.
Coorientador
Universidade Federal de Santa Catarina

RESUMO

A convergência entre robótica móvel e Internet das Coisas é central para a implementação dos Sistemas Ciber-Físicos. O *Robot Operating System 2* (ROS 2) consolidou-se como o padrão no desenvolvimento robótico e adotou o *Data Distribution Service* (DDS) como *middleware* de comunicação, garantindo entrega eficiente de mensagens. As mensagens nativas do ROS, porém, transmitem dados brutos de sensores sem o contexto físico e temporal necessário para sistemas de tomada de decisão autônoma. O conceito de *SmartData*, proposto por Fröhlich (2018), responde a essa lacuna ao encapsular cada leitura sensorial em um envelope que carrega, inseparavelmente do valor, sua unidade no Sistema Internacional, a origem espaço-temporal do dado e seu prazo de validade. Mais do que um formato, o SmartData associa cada dado a um transdutor, que abstrai um sensor, um atuador ou uma transformação de dados. Os interesses declarados entre SmartData formam uma rede pela qual os produtores respondem a consumidores remotos e os atuadores recebem comandos. Este trabalho projeta, implementa e avalia um *Gateway* semântico que exporta sensores e atuadores ROS 2 como transdutores SmartData e importa mensagens no formato SmartData para redes ROS 2. Um descritor YAML enumera os tópicos ROS e suas regras de decomposição, e um gerador em Python, executado no *build* antes da compilação, emite uma classe de transdutor por grandeza física e os objetos SmartData que a exportam. As mensagens de interesse, resposta e comando seguem o layout de mensagens TSTP e são difundidas em UDP multicast; um nó *proxy* reconstrói as mensagens ROS a partir das respostas e recebe também SmartData de outros produtores. A avaliação, conduzida em Linux nativo por hipóteses refutáveis, com repetições e intervalos de confiança, mostrou que a latência de tradução cresce linearmente com o número de campos da mensagem, cerca de 72 ns por campo, e fica em 26,8 µs no 99º percentil para uma varredura LiDAR de 360 feixes, abaixo do critério de 100 µs, adotado como um décimo do piso de latência do ROS 2 para cargas pequenas reportado por Kronauer et al. (2021). O envio das respostas, cada SmartData em um datagrama UDP próprio, domina o custo, mas é determinado pelo sistema operacional e pela interface de rede, e não pelo gateway, e o caminho completo de um publicador ROS a um assinante através da rede SmartData ficou abaixo de 1 ms no 95º percentil, com perda mediana nula, até 5.000 Hz. Em simulação no Gazebo, um robô TurtleBot3 Burger seguiu os comandos de velocidade recebidos pela rede SmartData com erro de regime inferior a $2 \times 10^{-3}\,\mathrm{m\,s^{-1}}$. Os resultados sustentam a viabilidade de integrar robôs ROS 2 a ecossistemas SmartData sem comprometer o desempenho do laço de controle.

Palavras-chave: ROS 2; SmartData; sistemas ciber-físicos; interoperabilidade semântica; transdutores; UDP multicast.

ABSTRACT

The convergence of mobile robotics and the Internet of Things is central to the implementation of Cyber-Physical Systems. The *Robot Operating System 2* (ROS 2) has become the standard in robotic development and adopted the *Data Distribution Service* (DDS) as its communication middleware, ensuring efficient message delivery. Native ROS messages, however, carry raw sensor data without the physical and temporal context required by autonomous decision-making systems. The *SmartData* concept, proposed by Fröhlich (2018), addresses this gap by encapsulating each sensor reading in an envelope that carries, inseparably from the value, its unit in the International System of Units, the spatio-temporal origin of the datum and its expiry. More than a format, SmartData binds each datum to a transducer, which abstracts a sensor, an actuator or a data transformation. Interests declared between SmartData form a network through which producers answer remote consumers and actuators receive commands. This work designs, implements and evaluates a semantic *gateway* that exports ROS 2 sensors and actuators as SmartData transducers and imports SmartData messages into ROS 2 networks. A YAML descriptor enumerates the ROS topics and their decomposition rules, and a Python generator, run during the build before compilation, emits one transducer class per physical quantity and the SmartData objects that export it. Interest, response and command messages follow the TSTP message layout and are disseminated over UDP multicast; a *proxy* node rebuilds the ROS messages from the responses and also receives SmartData from other producers. The evaluation, carried out on native Linux through refutable hypotheses, with repetitions and confidence intervals, showed that the translation latency grows linearly with the number of message fields, about 72 ns per field, and reaches 26,8 µs at the 99th percentile for a 360-beam LiDAR scan, below the 100 µs criterion, set as one tenth of the ROS 2 latency floor for small payloads reported by Kronauer et al. (2021). Sending the responses, each SmartData in its own UDP datagram, dominates the cost, but is determined by the operating system and the network interface rather than by the gateway, and the full path from a ROS publisher to a subscriber through the SmartData network stayed below 1 ms at the 95th percentile, with zero median loss, up to 5.000 Hz. In a Gazebo simulation, a TurtleBot3 Burger robot followed the velocity commands received through the SmartData network with a steady-state error below $2 \times 10^{-3}\,\mathrm{m\,s^{-1}}$. The results support the feasibility of integrating ROS 2 robots into SmartData ecosystems without compromising the performance of the control loop.

Keywords: ROS 2; SmartData; cyber-physical systems; semantic interoperability; transducers; UDP multicast.

LISTA DE FIGURAS

Figura 1 – Codificação da unidade SI em 32 bits 19
Figura 2 – Interface da classe SmartData 20
Figura 3 – Fluxo de dados entre o `gateway_node` e o `proxy_node` 25
Figura 4 – Latência de tradução em função do número de feixes 44
Figura 5 – Percentis da latência de tradução por número de iterações 45
Figura 6 – Taxa servida em função da taxa ofertada 46
Figura 7 – Latência por varredura em função do número de respostas 48
Figura 8 – Latência de ponta a ponta em função da taxa de publicação 49
Figura 9 – Velocidades comandadas e medidas no Gazebo 50

LISTA DE QUADROS

Quadro 1 – Mensagens SmartData na rede 32
Quadro 2 – Hipóteses, fatores e critérios de decisão dos experimentos 40
Quadro 3 – Síntese dos resultados frente aos critérios de decisão 51

LISTA DE TABELAS

Tabela 1 – Latência de tradução por mensagem (µs) 43
Tabela 2 – Percentis da latência de tradução por número de iterações (µs) . . . 44
Tabela 3 – Carga em laço aberto 46
Tabela 4 – Latência por varredura em função do número de respostas (µs) . . 47
Tabela 5 – Latência de ponta a ponta (µs) 48
Tabela 6 – Erro de regime e atraso de comando por degrau 50

LISTA DE CÓDIGOS

Código 1 – Declaração de sensores no descritor YAML 28
Código 2 – Estrutura de diretórios do pacote semantic_gateway 35

LISTA DE ABREVIATURAS E SIGLAS

CPS	*Cyber-Physical Systems* (Sistemas Ciber-Físicos)
DDS	*Data Distribution Service*
EPOS	*Embedded Parallel Operating System*
GCC	*GNU Compiler Collection*
IDL	*Interface Definition Language*
IMU	*Inertial Measurement Unit* (Unidade de Medição Inercial)
IoT	*Internet of Things* (Internet das Coisas)
LiDAR	*Light Detection and Ranging*
LISHA	Laboratório de Integração Software/Hardware
OMG	*Object Management Group*
PTP	*Precision Time Protocol*
QoS	*Quality of Service* (Qualidade de Serviço)
ROS	*Robot Operating System*
SDAV	*SmartData Autonomous Vehicle*
SI	Sistema Internacional de Unidades
TSTP	*Trustful Space-Time Protocol*
UDP	*User Datagram Protocol*
UFSC	Universidade Federal de Santa Catarina
YAML	*YAML Ain't Markup Language*

SUMÁRIO

1 INTRODUÇÃO 12
1.1 Problema de Pesquisa 12
1.2 Hipótese 13
1.3 Justificativa 13
1.4 Objetivos 13
1.4.1 Objetivo Geral 13
1.4.2 Objetivos Específicos 14
1.4.3 Escopo e Limites 14

2 FUNDAMENTAÇÃO TEÓRICA 15
2.1 Sistemas Ciber-Físicos 15
2.2 Sistemas de Tempo Real 15
2.3 Comunicação Publish-Subscribe 16
2.4 Robot Operating System 2 17
2.4.1 Modelo de Comunicação 17
2.4.2 Implementação por Camadas 17
2.4.3 Caracterização de Desempenho 18
2.4.4 Árvore de Transformações 18
2.5 O Conceito de SmartData 18
2.5.1 Estrutura do Envelope 19
2.5.2 Origem Espaço-Temporal e Validade 19
2.5.3 Transdutores Locais e Remotos 20
2.5.4 Interesses, Respostas e Comandos 21
2.5.5 Codificação em Pacotes de Rede 21

3 TRABALHOS RELACIONADOS 22
3.1 Desempenho de Middleware Robótico 22
3.2 Interoperabilidade Semântica em IoT 23
3.3 Implementações de SmartData 23

4 DESIGN DO SISTEMA 25
4.1 Visão Geral 25
4.2 Tópicos ROS como Transdutores 26
4.3 Linguagem Declarativa de Sensores 27
4.4 Geração de Código Prévia à Compilação 29
4.5 Modelo de Execução SmartData 30

4.6 Numeração de Dispositivos . . . 31
4.7 Mensagens na Rede . . . 32
4.7.1 Formato das Mensagens . . . 32
4.7.2 Transporte UDP Multicast . . . 32
4.7.3 Espelho em Tópico ROS . . . 33
4.8 O Lado Consumidor . . . 33

5 IMPLEMENTAÇÃO . . . 35
5.1 Estrutura do Pacote . . . 35
5.2 Codificação da Unidade SI . . . 36
5.3 Mensagens e Rede . . . 36
5.4 Objetos SmartData . . . 37
5.5 Caminho ROS para SmartData . . . 37
5.6 O Nó *Proxy* . . . 38
5.7 Geração de Código . . . 38

6 PLANO E EXECUÇÃO DE EXPERIMENTOS . . . 39
6.1 Ambiente Experimental . . . 39
6.2 Hipóteses . . . 40
6.3 Procedimentos . . . 40
6.3.1 Repetição e Estatísticas . . . 40
6.3.2 Latência de Tradução (H1) . . . 41
6.3.3 Carga em Laço Aberto (H2) . . . 41
6.3.4 Custo do Envio (H3) . . . 42
6.3.5 Latência de Ponta a Ponta (H4) . . . 42
6.3.6 Seguimento de Comandos (H5) . . . 43
6.4 Resultados . . . 43
6.4.1 Latência de Tradução (H1) . . . 43
6.4.2 Carga em Laço Aberto (H2) . . . 45
6.4.3 Custo do Envio (H3) . . . 47
6.4.4 Latência de Ponta a Ponta (H4) . . . 48
6.4.5 Seguimento de Comandos (H5) . . . 49
6.4.6 Síntese . . . 50
6.5 Discussão . . . 51

7 CONSIDERAÇÕES FINAIS . . . 53
7.1 Trabalhos Futuros . . . 53

REFERÊNCIAS . . . 55

1 INTRODUÇÃO

A evolução dos Sistemas Ciber-Físicos (*Cyber-Physical Systems*, CPS) e a consolidação da Internet das Coisas transformaram a forma como máquinas interagem com o mundo físico. Como argumenta Lee (2008), a essência dos CPS reside na orquestração integrada de computação, comunicação e controle, onde a dinâmica dos processos físicos exige o mesmo rigor formal que as computações lógicas. Nesse cenário, a robótica móvel deixou de operar como sistema isolado e tornou-se um componente ativo em ambientes inteligentes, integrada a infraestruturas de instrumentação, controle e analítica que extrapolam os limites do robô.

No domínio do desenvolvimento robótico, o *Robot Operating System* (ROS) estabeleceu-se como o padrão para a construção de sistemas complexos. O ROS 2 introduziu uma mudança arquitetural significativa ao substituir seu transporte próprio pelo *Data Distribution Service* (DDS), um padrão do *Object Management Group* (OMG) voltado a sistemas distribuídos de tempo real (MARUYAMA et al., 2016; MACENSKI et al., 2022). Paralelamente, no contexto de sistemas ciber-físicos em geral, de IoT industrial e cidades inteligentes a veículos autônomos, o conceito de *SmartData*, proposto por Fröhlich (2018), oferece uma abordagem orientada a dados que enriquece a informação bruta com metadados de semântica, localização e validade temporal, tornando cada leitura sensorial um objeto autocontido.

Apesar de coexistirem no mesmo nicho de aplicações ciber-físicas, os dois modelos de dados operam em níveis distintos de abstração. As mensagens do ROS expõem campos numéricos ou estruturas pouco anotadas, das quais o significado semântico, como unidade física, validade temporal e referencial espaço-temporal, é convenção do projeto, não dependente da mensagem. O SmartData, por sua vez, encapsula essas informações no próprio envelope, mas é difundido em um barramento próprio incompatível com o ferramental ROS. Serrano, Hauswirth e Soldatos (2015) caracterizam esse isolamento como um dos problemas centrais à interoperabilidade na Internet das Coisas, padrão que se reproduz na fronteira entre robótica e instrumentação ciber-física e motiva a tradução proposta neste trabalho.

1.1 PROBLEMA DE PESQUISA

A motivação deste trabalho está na ausência de um mecanismo direto e eficiente de tradução entre as mensagens operacionais do ROS 2 e o formato semanticamente enriquecido do SmartData. *Gateways* de aplicação genéricos existem, mas tipicamente introduzem latência incompatível com laços de controle robóticos ou perdem metadados durante a serialização. Formula-se, portanto, a seguinte questão de pesquisa: como

viabilizar a interoperabilidade semântica entre sistemas robóticos baseados em ROS 2 e plataformas de CPS orientadas a SmartData, mantendo os requisitos de tempo real e baixa sobrecarga de processamento?

1.2 HIPÓTESE

Este trabalho conjectura que é possível desenvolver um componente operando nativamente no ecossistema ROS 2 capaz de interceptar, enriquecer e retransmitir mensagens de sensores no formato SmartData, com uma latência de tradução, que compreende a extração dos campos, a conversão para unidades do SI, o enriquecimento com a origem espaço-temporal e o registro no SmartData, pequena o suficiente para não afetar o funcionamento dos demais subsistemas. Assume-se que, ao realizar a tradução dentro do próprio nó ROS, aproveitando as próprias informações do ecossistema para preencher os metadados de uma mensagem SmartData, elimina-se a necessidade de *brokers* externos complexos, preservando a integridade semântica dos dados.

1.3 JUSTIFICATIVA

A relevância desta pesquisa decorre da transição da arquitetura de sistemas robóticos para ecossistemas conectados e centrados em dados. A maturidade alcançada pelo ROS 2 (MACENSKI et al., 2022) resolveu problemas críticos de descoberta, escalabilidade e qualidade de serviço no transporte, mas não endereçou nativamente a compreensão semântica da informação: o *middleware* garante a entrega da mensagem, não que o receptor interprete inequivocamente a unidade física, o referencial de coordenadas ou a validade temporal do dado sem configuração prévia manual.

A lacuna semântica representa risco para a segurança e a confiabilidade de sistemas autônomos heterogêneos. Prestes et al. (2013) apontam que a falta de uma ontologia padronizada e acoplada aos dados em tempo de execução é causa frequente de falhas na composição de robôs de fabricantes ou módulos distintos. O SmartData mitiga esse risco ao acoplar inseparavelmente o valor numérico aos seus metadados de semântica, espaço e tempo (FRÖHLICH, 2018). Do ponto de vista de desempenho, executar a tradução dentro do próprio processo ROS, e não em um processo externo encadeado por mais uma travessia de serialização, preserva a viabilidade do caminho para laços de controle em tempo real (KRONAUER et al., 2021).

1.4 OBJETIVOS

1.4.1 Objetivo Geral

Projetar, implementar e validar experimentalmente um *Gateway* semântico para ROS 2 capaz de interceptar mensagens de sensores padrão e retransmiti-las em tempo real no formato SmartData, preservando a integridade semântica, espacial e temporal dos dados originais.

1.4.2 Objetivos Específicos

1. Definir um modelo de correspondência entre as mensagens de sensores e atuadores ROS e os transdutores do SmartData, expressível por meio de um arquivo declarativo.
2. Implementar o *Gateway* como nó nativo do ROS 2, aproveitando a árvore de transformações e o relógio do próprio ecossistema para enriquecer cada amostra com sua origem espaço-temporal.
3. Exportar cada grandeza como SmartData segundo o modelo de execução de transdutores, isto é, a forma como os SmartData anunciam, respondem a interesses e recebem comandos ao longo do tempo, proposto por Fröhlich (2018), com modos de operação, interesses orientados a eventos e periódicos, regiões espaço-temporais, validade e comandos, em mensagens com o *layout* do TSTP difundidas em UDP multicast, oferecendo como caminho secundário um espelhamento em tópico ROS para instrumentação e depuração no próprio ecossistema.
4. Implementar o lado consumidor, um *proxy* que reconstrói as mensagens ROS a partir das respostas recebidas, converte mensagens ROS em comandos e recebe SmartData de outros produtores da rede.
5. Caracterizar empiricamente a latência e a vazão da tradução e do envio, e verificar o caminho de ponta a ponta através da rede SmartData.

1.4.3 Escopo e Limites

A pesquisa delimita-se à camada de *software* de aplicação sobre ROS 2 Humble em Linux, tratando o DDS subjacente apenas como o transporte padrão do ROS, sem propor modificações ao *middleware*. A validação restringe-se a cenários simulados (gerador sintético e Gazebo) para garantir a reprodutibilidade. Mecanismos criptográficos previstos pelo SmartData estão fora do escopo deste trabalho, assim como as mensagens de controle do TSTP, responsáveis pela sincronização de relógios e pela gestão de chaves, o roteamento em múltiplos saltos e a supressão de respostas redundantes prevista pelo conceito original.

2 FUNDAMENTAÇÃO TEÓRICA

Este capítulo apresenta os conceitos sobre os quais o *Gateway* foi construído: Sistemas Ciber-Físicos como contexto da demanda por dados semanticamente ricos, os sistemas de tempo real e a comunicação *publish-subscribe* como fundamentos das exigências temporais e do modelo de comunicação envolvidos, o ROS 2 como plataforma robótica baseada em DDS, e o conceito de SmartData como abstração de dados e de transdutores em redes de sensores. Cada seção descreve as definições, propriedades e limitações que motivaram decisões de projeto adotadas nos capítulos posteriores.

2.1 SISTEMAS CIBER-FÍSICOS

Sistemas Ciber-Físicos são, na definição de Lee (2008), integrações de computação e processos físicos nas quais computadores embarcados e redes monitoram e controlam o mundo físico, comumente em laços de realimentação. O autor argumenta que a passagem do tempo, no mundo físico, é inevitável e a concorrência é intrínseca; ambas as propriedades, no entanto, não são tratadas pelas abstrações usuais, herdadas da computação de propósito geral. As consequências práticas incluem a dificuldade de raciocinar formalmente sobre latência, determinismo e ordenação em sistemas que mediam controle físico.

A demanda por confiabilidade em CPS é qualitativamente distinta da encontrada em *software* convencional. Em uma aplicação de processamento de texto, uma falha pode produzir desconforto, já em um sistema de controle automotivo, ela pode produzir danos físicos. Lee enfatiza que as decisões de projeto em CPS devem incorporar explicitamente o tempo e a concorrência como cidadãos de primeira classe, e não como propriedades emergentes da implementação. Esse argumento da base à adoção de mecanismos como o *Data Distribution Service* pelo ROS 2 e ao enriquecimento temporal dos pacotes SmartData no presente trabalho.

2.2 SISTEMAS DE TEMPO REAL

Um sistema de tempo real é, segundo Kopetz (2011), um sistema cujo comportamento correto depende não apenas do resultado lógico das computações, mas também do instante em que esses resultados são produzidos. Cada resultado está associado a um prazo (*deadline*), e a classificação do sistema decorre da consequência de perdê-lo: em sistemas de tempo real críticos (*hard*), a perda de um prazo pode causar danos graves, como no controle de um veículo; em sistemas não críticos (*soft*),

o resultado atrasado ainda tem utilidade, embora reduzida. O autor destaca, além disso, que uma observação do mundo físico tem precisão temporal limitada: a imagem que o computador mantém de uma grandeza física só é válida por um intervalo após a observação, findo o qual o estado real pode já ter mudado. Essa noção corresponde diretamente à validade carregada por cada SmartData.

A caracterização temporal de um sistema de tempo real envolve, por isso, mais do que o desempenho médio. A latência mede o intervalo entre um evento e a resposta do sistema, a variação da latência (*jitter*) mede a dispersão desse intervalo entre ativações, e o tempo de resposta no pior caso é o limite que precisa ser garantido para que os prazos sejam cumpridos (KOPETZ, 2011). Esses conceitos se aplicam diretamente ao *Gateway* proposto. Ele se insere no caminho entre os sensores e os consumidores dos dados, como controladores e sistemas de decisão, e todo o tempo gasto na tradução se soma à latência desse caminho e consome parte do prazo do laço de controle. Por esse motivo, o custo da tradução é avaliado em relação ao piso de latência do próprio ROS 2, e não em termos absolutos. Além disso, como sistemas operacionais de propósito geral produzem caudas longas na distribuição de latências, a avaliação reporta percentis elevados em vez da média. Por fim, a validade que o *Gateway* atribui a cada SmartData expressa a precisão temporal da observação, permitindo que o consumidor descarte um dado que, ao chegar, já não representa o estado do mundo físico.

2.3 COMUNICAÇÃO PUBLISH-SUBSCRIBE

No modelo de comunicação *publish-subscribe*, os consumidores registram junto a um serviço de eventos o seu interesse por determinados dados, enquanto os produtores publicam esses dados sem saber quem os receberá, cabendo ao serviço entregar cada publicação a todos os assinantes interessados (EUGSTER et al., 2003). De acordo com Eugster et al. (2003), o modelo se caracteriza por desacoplar as partes em três dimensões: no espaço, já que produtores e consumidores não precisam conhecer uns aos outros, no tempo, uma vez que não precisam participar da interação simultaneamente, e na sincronização, visto que nem a publicação nem a recepção bloqueiam o fluxo principal de execução. Os autores distinguem, além disso, duas formas de assinatura. Na assinatura por tópico, o consumidor escolhe um canal nomeado, e na assinatura por conteúdo, ele seleciona os dados por meio de predicados sobre os seus atributos.

As duas plataformas envolvidas neste trabalho adotam esse modelo, embora com formas de assinatura diferentes. O ROS 2, por meio do DDS, emprega tópicos identificados por nome e tipo de mensagem, cuja entrega é regulada por políticas de qualidade de serviço, como detalhado na Seção 2.4. Já no SmartData, um interesse

funciona como uma assinatura por conteúdo, pois o consumidor não nomeia um produtor nem um canal, e sim a grandeza física, por meio da unidade, além da região espaço-temporal e do período desejados, de modo que todo produtor que satisfaz esses critérios responde. Assim, o *Gateway* proposto opera na fronteira entre um modelo orientado a tópicos e um modelo orientado a conteúdo.

2.4 ROBOT OPERATING SYSTEM 2

O ROS é um conjunto de bibliotecas e ferramentas para o desenvolvimento de sistemas robóticos; o ROS 2 é seu sucessor, redesenhado para abordar limitações herdadas da primeira versão em segurança, tempo real e operação em redes não ideais (MACENSKI et al., 2022). A diferença arquitetural mais relevante para este trabalho é a substituição do transporte original do ROS 1 pelo DDS, um padrão do OMG para comunicação dirigida a dados, baseado no modelo publish/subscribe com descoberta distribuída e configurações de qualidade de serviço (QoS) (MARUYAMA et al., 2016).

2.4.1 Modelo de Comunicação

No ROS 2, uma aplicação é composta por processos, chamados nós (*nodes*), que se comunicam principalmente pela publicação e assinatura de tópicos, identificados por um nome e por um tipo de mensagem. Os tipos de mensagem são descritos em arquivos de interface e convertidos pelo próprio ROS para a *Interface Definition Language* (IDL) do DDS, a partir da qual são geradas estruturas de dados nativas em C++ e Python. Ao publicar, o nó serializa a mensagem e a entrega à camada DDS, que a encaminha a todos os assinantes do tópico de acordo com a política de qualidade de serviço (QoS) associada a ele. Além dos tópicos, o ROS 2 oferece serviços e ações para interações do tipo requisição e resposta, que não são utilizados neste trabalho.

A política de QoS é configurada por tópico e combina, entre outros, os seguintes parâmetros: o histórico, que define se apenas as últimas *N* mensagens são mantidas ou todas elas; a confiabilidade, que pode ser *reliable*, com retransmissão das mensagens perdidas, ou *best effort*, sem retransmissão; e a durabilidade, que determina se assinantes que chegam depois recebem mensagens já publicadas. O ROS 2 oferece perfis prontos para casos comuns. O perfil `SensorDataQoS`, voltado a sensores de alta frequência, usa entrega *best effort*, durabilidade volátil e histórico das últimas cinco mensagens, priorizando a amostra mais recente em vez da entrega de todas, já que uma leitura atrasada de sensor perde rapidamente o seu valor. O perfil `SystemDefaultsQoS`, por sua vez, não fixa nenhum desses parâmetros e adota os valores padrão da implementação de DDS em uso.

2.4.2 Implementação por Camadas

A pilha do ROS 2 segue uma estratificação clara (MACENSKI et al., 2022; MARUYAMA et al., 2016). Bibliotecas cliente como `rclcpp` expõem o modelo de *publish*/*subscribe* ao programador e delegam a comunicação a um *middleware interface module* (`rmw`), que por sua vez encapsula uma implementação concreta de DDS, como Fast DDS, Cyclone DDS ou Connext. A separação entre `rmw` e a implementação de DDS permite que o mesmo código de aplicação rode sobre fornecedores distintos.

2.4.3 Caracterização de Desempenho

A latência de comunicação do ROS 2 foi caracterizada em diversos estudos. Maruyama et al. (2016) mostraram que a sobrecarga da pilha do ROS 2 sobre o DDS bruto varia significativamente conforme o cenário, com valores da ordem de centenas de microssegundos para mensagens pequenas em processos distintos. Kronauer et al. (2021) estenderam essa análise para sistemas com múltiplos nós e mostraram que a pilha do ROS 2 pode acrescentar até 50% de sobrecarga à latência do DDS bruto para mensagens pequenas. A latência de ponta a ponta, isto é, o intervalo entre a publicação de uma mensagem e a sua entrega ao assinante, ficou tipicamente entre 1 ms e 3 ms no 95º percentil, para cargas de 128 B a 500 kB em configurações representativas. Esses dados são a referência mais diretamente comparável à sobrecarga adicional que um tradutor para SmartData deve evitar reintroduzir.

2.4.4 Árvore de Transformações

Cada sensor de um robô mede o mundo a partir do seu próprio ponto de vista, isto é, em relação a um quadro de referência (*frame*) definido por uma posição e uma orientação. O subsistema `tf2` do ROS mantém, ao longo do tempo, as transformações que relacionam esses quadros, organizadas em uma árvore cuja raiz é um quadro global, como o mapa ou a origem da odometria. Uma consulta à árvore retorna, para um instante específico, a transformação entre dois quadros quaisquer, obtida pela composição das transformações ao longo do caminho que os liga. Essa estrutura é a fonte canônica para preencher a origem espacial do SmartData a partir do `frame_id` declarado no cabeçalho das mensagens de sensor.

2.5 O CONCEITO DE SMARTDATA

O SmartData foi proposto por Fröhlich (2018) como a abstração primária usada por aplicações para interagir com o mundo físico em uma rede de sensores e atuadores. Trata-se de um dado enriquecido com metadados suficientes para ser autocontido quanto à semântica, à localização espacial, ao instante de produção e à confiabili-

dade. Conceitualmente, ler uma instância de SmartData implica amostrar o transdutor associado, e escrever em uma instância define um novo *setpoint* para um atuador.

2.5.1 Estrutura do Envelope

Cada envelope SmartData carrega, além do valor numérico, uma unidade codificada como inteiro de 32 bits que descreve a dimensão SI da grandeza sensoriada, as coordenadas (x, y, z) da origem espacial, um *timestamp* de alta resolução, um prazo de validade (*expiry*) relativo ao instante de produção e um código de autenticação. A confiabilidade do dado é expressa por dois campos adicionais: a incerteza associada à medição e o grau de confiança, de 0 a 100%, na localização informada. A unidade é codificada com um bit de tipo (físico/digital), dois bits para o tipo numérico, dois bits para um modificador (direto, inverso, logarítmico) e nove campos de três bits para os expoentes das sete dimensões fundamentais do SI mais radiano e esterradiano, conforme a Figura 1. Cada expoente é armazenado somado de 4, de modo que os três bits representam valores de -4 a +3.

Figura 1 – Codificação da unidade SI em 32 bits

31	30–29	28–27	26–24	23–21	20–18	17–15	14–12	11–9	8–6	5–3	2–0
1	NUM	MOD	sr+4	rad+4	m+4	kg+4	s+4	A+4	K+4	mol+4	cd+4

Fonte: adaptado de Fröhlich (2018).

Essa codificação produz, por exemplo, a constante `METER` com expoente +1 na dimensão de comprimento e zero nas demais; `METER_PER_SEC_SQ` com expoentes +1 em comprimento e -2 em tempo, descrevendo aceleração; e `VOLT` como composto das dimensões de massa, comprimento, tempo e corrente segundo a definição formal. Portanto, a semântica é intrínseca ao identificador de unidade, sem necessidade de tabelas de tradução externas.

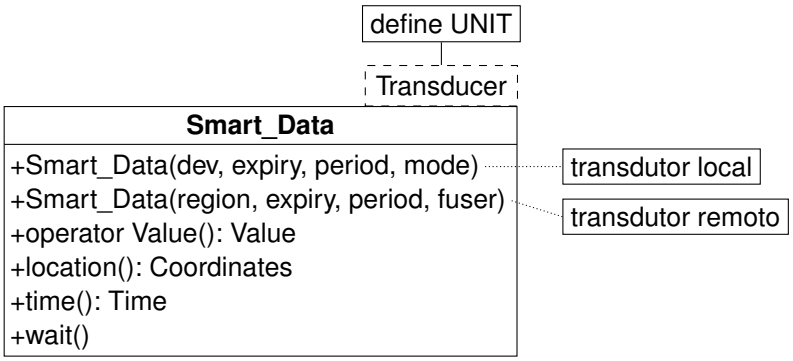
2.5.2 Origem Espaço-Temporal e Validade

O *timestamp* de origem é expresso em microssegundos contados a partir de uma época absoluta, por padrão, a época UNIX. A validade relativa permite que o consumidor avalie, a qualquer instante, se o dado ainda é semanticamente utilizável, desde que os relógios do produtor e do consumidor estejam sincronizados, já que a comparação envolve o *timestamp* atribuído pelo primeiro e o relógio local do segundo: um valor lido após o vencimento foi produzido em um ponto do passado em que as condições físicas observadas podem já não corresponder ao estado atual do sistema. Em Fröhlich et al. (2018), a validade é discutida como um substituto operacional para os prazos de tarefas em escalonadores de tempo real, deslocando o raciocínio temporal da execução para os dados.

2.5.3 Transdutores Locais e Remotos

A ligação entre um objeto SmartData e o mundo físico é feita por uma classe de transdutor, passada como parâmetro de tipo, que declara a constante `UNIT` segundo a codificação descrita acima (FRÖHLICH, 2018). Um SmartData cujo transdutor declara Kelvin representa a grandeza temperatura e pode abstrair tanto um sensor quanto um atuador, como um condicionador de ar. Instâncias de SmartData são criadas por um de dois construtores, mostrados na Figura 2.

Figura 2 – Interface da classe SmartData



Fonte: adaptado de Fröhlich (2018).

O primeiro construtor abstrai transdutores locais e recebe, além do transdutor, um identificador de dispositivo (`dev`), a validade, o modo de operação e, opcionalmente, um período. O identificador `dev` diferencia múltiplas instâncias do mesmo transdutor disponíveis em um nó. Com período diferente de zero, o transdutor é amostrado periodicamente; caso contrário, é consultado a cada acesso ao SmartData, ou sinaliza dados novos por interrupção, caso em que o acesso bloqueia até a sinalização. Três modos de operação são definidos: no modo *privado*, o SmartData pertence ao processo que o criou e não é anunciado na rede, no modo *anunciado*, ele pode ser monitorado remotamente e no modo *comandado*, além de anunciado, pode ser controlado remotamente. O modo comandado é o que distingue o conceito de outras infraestruturas de sensoriamento: em vez de enviar comandos diretamente a atuadores, a aplicação escreve um novo valor, que funciona como *setpoint*, isto é, como a referência a ser atingida pela grandeza na região designada. Cabe então a um sistema de controle associado aos transdutores dessa região comparar essa referência com o estado medido e acionar os atuadores para alcançá-la.

O segundo construtor cria *proxies* locais de transdutores remotos. Seu primeiro parâmetro é uma região espaço-temporal de interesse, $Region(x, y, z, r, t_0, t_f)$, em que (x, y, z) é o centro da região, r o seu raio e $[t_0, t_f]$ o intervalo de interesse; seguem-se a validade, o período e, opcionalmente, uma função de fusão. A instanciação de um

SmartData remoto implica o anúncio de um interesse pela grandeza física da unidade do transdutor na região dada.

2.5.4 Interesses, Respostas e Comandos

Os SmartData que satisfazem os critérios de um interesse respondem a ele conforme o período e a função de fusão do interesse (FRÖHLICH, 2018). Período zero indica que todas as atualizações dos nós remotos devem ser reportadas, definindo uma operação orientada a eventos (*event-driven*), já outros valores exigem uma resposta a cada período, durante o intervalo de tempo definido por $[t_0, t_f]$, definindo uma operação disparada por tempo (*time-triggered*). Quando SmartData distintos usam o mesmo transdutor com períodos diferentes, a amostragem ocorre no máximo divisor comum dos períodos, embora cada SmartData interessado continue a observar o seu período original. Sem função de fusão, um SmartData responde ao interesse apenas se ainda não ouviu a resposta de outro nó, e o valor do *proxy* é o da última resposta recebida, e com função de fusão, todos os transdutores que satisfazem o critério respondem, e cada nova resposta é entregue à função, cujo retorno define o valor do SmartData.

A validade (*expiry*) indica por quanto tempo, a partir do instante de produção, o valor de um SmartData continua representando adequadamente a grandeza observada. Ela é expressa de forma relativa e renovada a cada atualização, de modo que um dado produzido no instante t com validade e pode ser usado até $t + e$. Depois disso, o valor é considerado vencido, e acessá-lo dispara uma nova atualização do transdutor (FRÖHLICH, 2018). Nos *proxies*, a validade é uma informação do cliente: vários SmartData que representam o mesmo transdutor remoto podem adotar validades distintas. Os nós que exportam SmartData anunciados gerenciam as respostas de modo a atender a todos os interesses e usam, no pacote de rede, a menor das validades envolvidas.

2.5.5 Codificação em Pacotes de Rede

A codificação dos atributos do SmartData em um pacote de rede depende do protocolo de comunicação. A implementação de referência foi construída sobre o *Trustful Space-Time Protocol* (TSTP) (RESNER et al., 2015), um protocolo *cross-layer* que provê nativamente os conceitos de tempo, localização e segurança exigidos pelo SmartData: os campos de origem, validade e autenticação são definidos e usados pelo próprio TSTP e apenas reaproveitados pela construção SmartData. Nessa implementação, o comprimento dos campos se adapta ao cenário de implantação, e campos adicionais descrevem a codificação e a versão do protocolo de cada pacote (FRÖHLICH, 2018).

3 TRABALHOS RELACIONADOS

A literatura adjacente a este trabalho organiza-se em três frentes: caracterizações de desempenho do ROS 2 e do DDS, propostas de interoperabilidade semântica para a Internet das Coisas e implementações do conceito de SmartData em pilhas de comunicação industriais. Esta seção discute cada frente e posiciona a contribuição da presente monografia em relação a elas.

3.1 DESEMPENHO DE MIDDLEWARE ROBÓTICO

Maruyama et al. (2016) produziram a primeira caracterização sistemática do desempenho do ROS 2 sobre DDS, ainda em uma versão alfa, comparando-o ao ROS 1 e a três implementações de DDS: Connext, OpenSplice e FastRTPS. Os autores mediram latência de ponta a ponta, vazão, número de *threads* e consumo de memória. Para isso, variaram o tamanho da mensagem de 256 B a 4 MB e a disposição dos nós, que podiam estar em máquinas distintas, na mesma máquina ou no mesmo processo. Quanto à QoS, foram definidos dois perfis. O perfil confiável usa histórico completo, entrega *reliable* e durabilidade *transient local*, de modo que assinantes que chegam depois ainda recebem as mensagens anteriores. O perfil de melhor esforço, por sua vez, mantém apenas a última mensagem, não retransmite perdas e usa durabilidade volátil. Os autores avaliaram ainda variações na profundidade do histórico, com 1, 10 e 100 mensagens.

Na mesma máquina, a sobrecarga do ROS 2 em relação ao ROS 1 mostrou-se não desprezível, e os autores a atribuem às duas conversões de dados entre o ROS e o DDS e ao processamento do próprio DDS. A latência permaneceu praticamente constante enquanto a mensagem cabia em um único datagrama UDP, de até 64 kB, e cresceu abruptamente a partir do momento em que passou a ser fragmentada. A política confiável acrescentou latência em troca da retransmissão e da tolerância a assinantes tardios, ao passo que a profundidade do histórico não teve efeito sobre a latência. A implementação de DDS, por outro lado, mostrou-se determinante, já que o OpenSplice obteve as menores latências locais entre as avaliadas. Além disso, o ROS 2 distribuiu as mensagens entre múltiplos assinantes de forma mais equilibrada que o ROS 1. Os autores concluem, portanto, que o desempenho do ROS 2 depende fortemente do par `rmw`/DDS e da configuração de QoS, que precisam ser ajustados ao cenário de aplicação.

Kronauer et al. (2021) estenderam a análise para sistemas com múltiplos nós e quantificaram a sobrecarga adicionada pela pilha do ROS 2 em relação ao DDS bruto. Encontraram que, para mensagens pequenas, da ordem de 128 B, o ROS 2 acrescenta

até 50% à latência das chamadas diretas ao DDS, e que, além do próprio DDS, a maior parcela dessa latência vem do atraso de notificação do `rclcpp`, isto é, do intervalo entre a chegada da mensagem ao DDS e a sua entrega ao *callback* pelo executor. Para mensagens grandes, a fração relativa da sobrecarga encolhe e o gargalo passa a ser o próprio DDS.

A relevância desses resultados para o presente trabalho é direta. Um tradutor que se interponha entre a mensagem ROS original e o consumidor SmartData não pode replicar dentro do seu processo a mesma sobrecarga que a pilha já paga. As decisões de projeto descritas no Capítulo 4, assinatura do tipo concreto de cada mensagem na própria pilha `rclcpp`, ausência de introspecção em tempo de execução e barramento UDP binário sem intermediação por DDS, são respostas diretas a essa preocupação.

3.2 INTEROPERABILIDADE SEMÂNTICA EM IOT

Serrano, Hauswirth e Soldatos (2015) discutem a fragmentação do ecossistema de Internet das Coisas em silos verticais que não interoperam e propõem uma pilha de orquestração baseada em *middleware* semântico, o OpenIoT-VDK. A estratégia dos autores é introduzir camadas de anotação, filtragem e agregação em um *broker* central que media o acesso a sensores virtuais e físicos. A abordagem é eficaz em cenários de coleta de dados em escala urbana, mas é incompatível com a latência exigida por laços de controle robótico, pois cada amostra atravessa processos adicionais antes de chegar ao consumidor.

O presente trabalho herda a motivação, interoperabilidade semântica entre subsistemas heterogêneos, mas adota uma abordagem arquitetonicamente oposta: a tradução ocorre dentro do mesmo processo que assina o tópico ROS, sem *brokers* externos, e o consumidor SmartData recebe o envelope diretamente do barramento, sem intermediação adicional. Essa escolha é motivada pela latência, pois cada processo ou *broker* intermediário acrescentaria ao menos uma passagem adicional pela rede ou pela comunicação entre processos no caminho entre o sensor e o laço de controle. A solução é mais restrita em escopo (não trata de agregação nem de consulta histórica), mas adequada ao domínio robótico.

3.3 IMPLEMENTAÇÕES DE SMARTDATA

O conceito de SmartData foi originalmente proposto e implementado por Fröhlich (2018) no contexto do *Embedded Parallel Operating System* (EPOS) e do *Trustful Space-Time Protocol* (TSTP), uma pilha de rede própria para sensores embarcados. A implementação original endereçava nós embarcados de baixíssimo consumo energético, com o EPOS abstraindo transdutores como objetos C++ e o TSTP provendo entrega segura e sincronizada (RESNER et al., 2015). Essa origem moldou a estrutura

do envelope: a presença do campo de validade, a codificação compacta da unidade e o comprimento adaptável dos campos do pacote refletem decisões orientadas a redes de sensores com restrições severas de banda.

Em Fröhlich et al. (2018), o conceito é estendido para o projeto de CPS completos, com discussão sobre escalonamento orientado a dados, no qual tarefas são executadas para produzir os SmartData demandados antes que expirem. Essas implementações compartilham a estratégia de produzir SmartData diretamente em transdutores embarcados, abaixo de qualquer pilha de aplicação.

A implementação SDAV (*SmartData Autonomous Vehicle*) (LISHA/UFSC, 2026), desenvolvida no contexto do projeto de veículos autônomos do LISHA, reproduz em C++ as mensagens de interesse, resposta e comando do TSTP e as difunde em UDP multicast, levando o SmartData a computadores de propósito geral. O presente trabalho a adota como referência para o formato das mensagens, porém adaptando alguns elementos conforme necessário: o sistema de suporte à execução do *Gateway* foi escrito para as necessidades de um nó ROS, como o relógio do próprio ROS, inclusive sob tempo simulado, e a operação sobre um transporte sem garantia de entrega. Como a validade é avaliada comparando estampas de tempo atribuídas por nós distintos, esse relógio precisa estar sincronizado com o dos demais participantes da rede SmartData, questão discutida na Seção 4.1.

A contribuição original deste trabalho complementa essas implementações ao oferecer a ponte que faltava: a entrada do ecossistema robótico. Enquanto as implementações anteriores produziam SmartData em transdutores embarcados e os consumiam em aplicações industriais, este trabalho posiciona o gateway no ponto em que mensagens ROS já existentes precisam ser elevadas semanticamente para alcançar consumidores SmartData, sem demandar reescrita dos sensores ou do ferramental robótico, e, no sentido inverso, no ponto em que SmartData produzidos na rede precisam chegar a aplicações ROS.

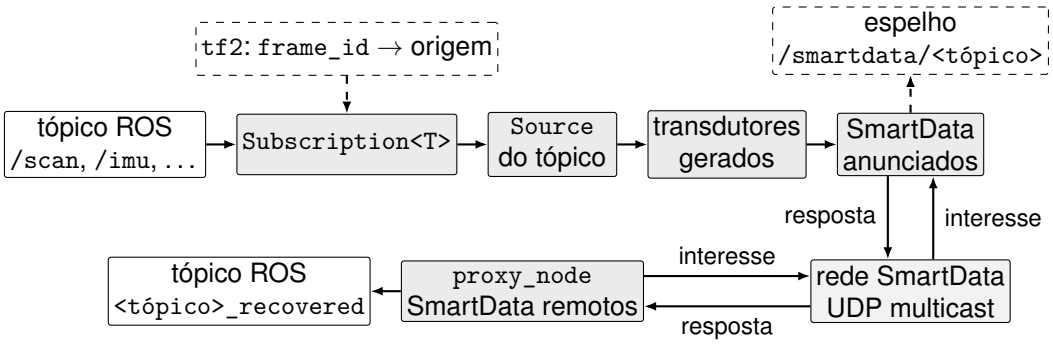
4 DESIGN DO SISTEMA

Este capítulo descreve a arquitetura do *Gateway* semântico e as decisões de projeto que conduziram à sua forma final. A apresentação parte de uma visão geral do fluxo de dados, apresenta o modelo que trata os tópicos ROS como transdutores SmartData, detalha a linguagem declarativa que descreve cada tópico, discute a estratégia de geração de código, o modelo de execução SmartData adotado, a numeração de dispositivos e o formato das mensagens na rede, e termina com o lado consumidor da solução.

4.1 VISÃO GERAL

O Gateway é composto por dois executáveis ROS 2 e por um sistema de suporte à execução SmartData próprio, independente do *middleware* do ROS e baseado em *sockets* UDP multicast. O primeiro executável, `gateway_node`, exporta o robô para a rede SmartData: assina os tópicos de sensores declarados no descritor, entrega cada mensagem recebida aos transdutores que extraem dela as grandezas físicas e mantém um SmartData por instância de transdutor. Esses objetos são anunciados na rede e respondem aos interesses que recebem, os atuadores, por sua vez, recebem comandos da rede e publicam o *setpoint* resultante no tópico ROS correspondente. O segundo executável, `proxy_node`, é o lado consumidor: instancia SmartData remotos, isto é, *proxies* locais de transdutores remotos (Seção 2.5.3), que anunciam interesses, recebem as respostas e republicam as mensagens ROS reconstruídas, convertem mensagens ROS em comandos e recebem também SmartData de outros produtores da rede.

Figura 3 – Fluxo de dados entre o `gateway_node` e o `proxy_node`



Fonte: elaborada pelo autor.

A Figura 3 resume o caminho percorrido por uma mensagem. Uma mensagem ROS chega à `Subscription` criada pelo gateway para o tipo de mensagem do tópico;

o *callback* correspondente consulta a árvore `tf2` para resolver a origem espacial associada ao `frame_id` da mensagem e a entrega aos transdutores daquele tópico. Cada transdutor extrai o seu campo, converte-o para o SI e notifica o SmartData ao qual está ligado, que responde aos interesses associados com uma mensagem de resposta difundida em UDP multicast. No `proxy_node`, as respostas que compartilham a mesma origem temporal são reagrupadas na mensagem ROS original. Opcionalmente, toda atualização local é também espelhada em um tópico ROS `/smartdata/<tópico>` do tipo `semantic_gateway/msg/SmartData`.

O sistema de suporte à execução segue a implementação SDAV, utilizada no projeto de veículos autônomos do LISHA, como referência de formato, mas foi adaptado para as necessidades do gateway. Duas diferenças orientaram essa escolha: o relógio é injetado, para que o gateway opere sob qualquer fonte de tempo adotada pelo ROS, como o tempo simulado do Gazebo, e os interesses são tratados como estado temporário (*soft state*), isto é, cada interesse deixa de valer se não for renovado periodicamente pelo consumidor, pois o transporte UDP não oferece a confiabilidade do TSTP e um interesse ou um cancelamento perdido não pode deixar o produtor em um estado incorreto (Seção 4.5).

O relógio injetado, contudo, precisa estar sincronizado com o dos demais participantes da rede SmartData, pois é com ele que o *Gateway* estampa os dados que produz e avalia a validade dos que recebe. Sem uma base de tempo comum, a comparação entre o instante de produção de um dado e o relógio de quem o consome deixa de ter significado, e com ela o próprio conceito de validade. Nos experimentos deste trabalho, todos os nós executam no mesmo computador, de modo que essa condição é satisfeita por construção. No cenário sintético, todos leem o relógio do sistema, e, no cenário Gazebo, todos seguem o mesmo tempo simulado publicado no tópico `/clock`. Em uma implantação com nós SmartData embarcados, por outro lado, o relógio do ROS teria de ser sincronizado com o da rede, por exemplo pelo mecanismo de sincronização do TSTP ou pelo *Precision Time Protocol* (PTP), de modo que ambos os lados compartilhem a mesma base de tempo.

4.2 TÓPICOS ROS COMO TRANSDUTORES

O ponto de partida do projeto é tratar cada grandeza física carregada por um tópico ROS como um transdutor no sentido de Fröhlich (2018). Uma mensagem `Imu` carrega duas grandezas, a aceleração linear, em $\mathrm{m\,s^{-2}}$, e a velocidade angular, em $\mathrm{rad\,s^{-1}}$, cada uma com três componentes. O gateway define, para ela, duas classes de transdutor, cada uma declarando a sua `UNIT`, e três instâncias de cada classe, distinguidas pelo parâmetro `dev` do construtor de transdutores locais (Seção 2.5.3), que é exatamente o papel que o conceito original atribui a esse parâmetro. Analogamente,

um `LaserScan` define uma única classe de comprimento, com uma instância por feixe.

Os transdutores do gateway são ativos. Não há *hardware* a amostrar, a chegada de uma mensagem no tópico assinado cumpre o papel da interrupção que sinaliza um dado novo, e cada instância extrai o seu campo, converte-o para o SI e notifica o SmartData ao qual está ligada. Diferentemente de um transdutor embarcado, cuja localização é a do próprio nó, um sensor ROS não está no nó que executa o gateway. A origem espacial de cada amostra é, por isso, resolvida na árvore `tf2` a partir do `frame_id` da mensagem, no instante indicado por `header.stamp`, e expressa em um quadro de referência configurável (`target_frame`), que faz o papel do sistema cartesiano centrado no gateway previsto pelo conceito original. Quando a transformação no instante exato ainda não está disponível, usa-se a última conhecida. A confiança da localização, de 100% para a transformação exata, 50% para a última conhecida e 0% quando nenhuma existe, é transmitida no campo correspondente da mensagem, de modo que o consumidor sabe quanto confiar na posição sem que a amostra seja descartada. Esses três níveis foram definidos de forma arbitrária neste trabalho. Uma política mais fundamentada poderia derivar a confiança da idade da última transformação conhecida e da velocidade do quadro em relação à referência, reduzindo-a à medida que cresce o deslocamento possível desde então.

Cada instância é exportada por um SmartData de transdutor local, sensores no modo anunciado e atuadores no modo comandado. Um atuador como o `/cmd_vel`, do tipo `Twist`, segue a mesma decomposição, com duas classes, velocidade linear e angular, e seis instâncias, uma por componente, cuja escrita compõe o *setpoint* publicado no tópico.

4.3 LINGUAGEM DECLARATIVA DE SENSORES

A configuração do *Gateway* é especificada em um arquivo YAML que declara, para cada tópico a ser exportado, o tipo de mensagem, o padrão de decomposição e parâmetros específicos da decomposição. Três padrões cobrem os tipos relevantes da biblioteca `sensor_msgs`:

Escalar Uma mensagem ROS carrega exatamente uma grandeza, que define uma classe de transdutor com uma instância. O campo `value_field` aponta para o atributo numérico a ser extraído, e um `value_transform` linear opcional permite ajustar a unidade (por exemplo, conversão de graus Celsius para Kelvin pela aplicação de $a \cdot v + b$). A transformação inversa é gerada automaticamente para o caminho de reconstrução.

Vetor Uma mensagem ROS carrega N valores da mesma grandeza, que definem uma classe de transdutor com uma instância por elemento de um campo do tipo

arranjo (*array*) da mensagem. O caso prototípico é o `LaserScan`, cuja distância de cada feixe é exportada por um SmartData independente. Filtros opcionais (`finite`, `finite_within`) descartam feixes inválidos ou fora da faixa declarada do sensor; um feixe descartado não produz amostra, e o SmartData correspondente mantém o valor anterior até que ele expire.

Composto Uma mensagem ROS é decomposta em grupos de campos, cada grupo uma classe de transdutor com a sua própria unidade e uma instância por campo. O caso prototípico é o `Imu`, cujos três componentes de aceleração linear formam um grupo na unidade $\mathrm{m\,s^{-2}}$ e os três componentes de velocidade angular formam outro grupo em $\mathrm{rad\,s^{-1}}$.

Os três padrões valem também no caminho inverso, em que o *proxy* reconstrói a mensagem ROS a partir dos SmartData recebidos. Os atuadores, por sua vez, aceitam os padrões escalar e composto. Um `Twist`, por exemplo, é declarado como composto, com um grupo de velocidade linear e outro de velocidade angular, e a escrita de cada componente compõe a mensagem publicada no tópico do atuador.

Cada tópico declara também um prazo de validade (`expiry_ms`) e, opcionalmente, um período de amostragem (`period_ms`) e o modo de operação, anunciado por padrão ou privado. A estampa de tempo de cada amostra é derivada do campo de cabeçalho ROS indicado por `stamp_field`, e a origem espacial vem da consulta `tf2` ao quadro indicado por `frame_field`. A seção `actuators` declara os atuadores com os mesmos padrões, acrescidos do quadro `tf2` que localiza o atuador (`frame`) e de uma janela de coalescência (`coalesce_ms`), discutida na Seção 4.5. A seção `imports`, apresentada na Seção 4.8, declara SmartData produzidos fora do gateway. O Código 1 mostra a declaração de dois sensores.

Código 1 – Declaração de sensores no descritor YAML

```yaml
sensors:
  - topic: /temperature
    type: sensor_msgs/msg/Temperature
    decomposition: scalar
    value_field: temperature
    value_transform: { kind: linear, a: 1.0, b: 273.15 }
    unit: kelvin
    expiry_ms: 500

  - topic: /imu
    type: sensor_msgs/msg/Imu
    decomposition: composite
    groups:
      - name: linear_acceleration
        unit: meter_per_sec_sq
```

```
16          fields: [linear_acceleration.x, linear_acceleration.y,
17                   linear_acceleration.z]
18        - name: angular_velocity
19          unit: rad_per_sec
20          fields: [angular_velocity.x, angular_velocity.y,
21                   angular_velocity.z]
22      expiry_ms: 10
```

Fonte: elaborada pelo autor.

4.4 GERAÇÃO DE CÓDIGO PRÉVIA À COMPILAÇÃO

A tradução entre as mensagens ROS 2 e os transdutores é produzida por um gerador de código executado durante o *build*, antes da compilação. O gerador, escrito em Python, lê o descritor YAML em duas etapas. Na primeira, constrói um modelo intermediário com os tópicos, as classes de transdutor e as importações declaradas, e valida a configuração: unidades desconhecidas, tópicos declarados duas vezes, transformações não inversíveis e dispositivos importados que colidem com os blocos locais interrompem a geração com uma mensagem que identifica a entrada do descritor responsável. Na segunda etapa, cada elemento do modelo é emitido como código C++.

No código emitido, cada acesso a campo aparece de forma literal (por exemplo, `msg.linear_acceleration.x`), cada `Subscription` é instanciada com o seu tipo estático e cada unidade é uma constante de compilação. O custo por mensagem reduz-se, assim, a uma sequência de acessos diretos aos campos, que o compilador pode otimizar como qualquer outro código C++. Pela mesma razão, os erros que o gerador não detecta são detectados pelo compilador, como um campo inexistente ou um tipo incompatível no descritor resulta em erro de compilação, e não em falha durante a operação do robô. Em contrapartida, alterar o descritor exige regenerar os *headers* e recompilar os executáveis, o que é compatível com o uso pretendido, pois o descritor faz parte da configuração do robô e é alterado antes do *deploy*. O gerador emite três *headers* C++, cada um responsável por uma fatia bem delimitada do sistema:

`parsed_transducers.hpp` Contém uma classe de transdutor por grandeza de cada tópico, derivada do modelo `Transducer<UNIT>` do sistema de suporte à execução. Cada classe declara a sua unidade, o seu tipo (sensor, atuador ou ambos), o fato de ser ativa e o início do seu bloco de dispositivos. Nos sensores, o método `update` usa o deslocamento do dispositivo para escolher o seu campo, com acesso direto (por exemplo, `msg.linear_acceleration.x`), aplica a transformação e os filtros declarados e registra a amostra, já nos atuadores, o método `actuate` faz o caminho inverso.

parsed_gateway.hpp Contém a classe `Gateway`, usada pelo `gateway_node`, que cria as `rclcpp::Subscription<T>`, os pontos de distribuição de cada tópico aos seus transdutores, os publicadores dos atuadores e um SmartData por instância de transdutor. As instâncias dos transdutores vetoriais são criadas quando a primeira mensagem revela o número de elementos do sensor.

parsed_proxies.hpp Contém a classe `Proxies`, usada pelo `proxy_node`, que reúne os SmartData remotos de cada classe de transdutor, os estados de reconstrução das mensagens, cada um definido para o tipo da mensagem alvo e com a inversa do `value_transform` embutida, as assinaturas que convertem mensagens em comandos e os SmartData importados de outros produtores.

A geração é integrada ao *build* por um `add_custom_command` do CMake que declara o descritor e o próprio gerador como dependências, de modo que os *headers* são regenerados sempre que um dos dois é modificado e, em seguida, compilados junto com os executáveis. Toda a tradução é, portanto, resolvida estaticamente, os executáveis não leem o descritor durante a execução, e a única forma da configuração presente no compilado é o código gerado.

4.5 MODELO DE EXECUÇÃO SMARTDATA

O sistema de suporte à execução do gateway implementa o modelo de execução descrito na Seção 2.5.4. Um SmartData anunciado permanece em silêncio até receber um interesse compatível, com a mesma unidade, dispositivo igual ao seu ou curinga, região que contenha a origem da sua última amostra e intervalo de tempo ainda aberto. Cada interesse aceito é guardado como uma associação, e o SmartData responde imediatamente com o seu valor corrente, para que o consumidor não precise esperar a próxima amostra, a partir daí, responde a cada atualização do transdutor, para os interesses orientados a eventos, ou a cada período, para os interesses periódicos. Interesses com períodos distintos são atendidos por um único temporizador no máximo divisor comum dos períodos. A validade enviada em cada resposta é a menor entre a do SmartData e as dos interesses associados, e um SmartData que ainda não recebeu nenhuma amostra não responde, para não difundir um valor fictício.

No TSTP, a entrega dos interesses é garantida pelo próprio protocolo. Sobre UDP, que não oferece essa garantia, um interesse perdido ou um consumidor iniciado antes do produtor quebrariam a associação de forma permanente. Adotou-se, por isso, um tratamento dos interesses como estado temporário (*soft state*): o consumidor reanuncia cada interesse a cada segundo, e o produtor desfaz as associações não renovadas em 3,5 s, o que também encerra o envio para consumidores que deixaram a rede sem revogar os seus interesses. O prazo de 3,5 s equivale a três períodos e

meio de renovação. Assim, uma associação sobrevive à perda de até três reanúncios consecutivos, e o meio período adicional absorve variações no instante de envio. Ambos os valores são parâmetros da `Network` e representam um compromisso entre a tolerância a perdas e a rapidez com que o produtor deixa de atender um consumidor que saiu da rede. A revogação explícita continua sendo enviada quando um *proxy* é destruído.

Os comandos seguem o caminho inverso. Escrever um valor em um SmartData remoto envia uma mensagem de comando com a região, a validade e o novo valor, os SmartData comandados da unidade correspondente cuja posição está na região repassam o valor ao transdutor, que o escreve no seu componente do *setpoint* e o reporta como nova amostra, de modo que o estado do atuador também pode ser observado na rede. Quando a mensagem ROS de um atuador tem vários campos, cada campo é exportado como um SmartData próprio, e a escrita remota de vários deles chega ao gateway como comandos independentes, um por campo. Publicar um *setpoint* a cada comando entregaria ao atuador uma rajada de mensagens parcialmente atualizadas, nas quais apenas parte dos campos já tem o novo valor. Por isso, o primeiro comando recebido abre uma janela de coalescência, de 2 ms por padrão e configurável por tópico no descritor: os comandos que chegam dentro dela atualizam os seus campos, os demais mantêm o último valor, e ao fim da janela uma única mensagem é publicada. Com janela nula, cada comando é publicado imediatamente, o que basta para atuadores de um único campo.

4.6 NUMERAÇÃO DE DISPOSITIVOS

A rede transporta SmartData de todos os tópicos em um único grupo multicast, e duas classes de transdutor podem compartilhar a mesma unidade, como a velocidade angular da IMU e a do `/cmd_vel`. O gerador aloca, por isso, uma faixa disjunta do espaço de dispositivos a cada classe de transdutor declarada, sequencialmente, com tamanho fixo de 2^{16} endereços. A classe de índice i recebe a faixa $[i \cdot 2^{16}, (i+1) \cdot 2^{16})$, e a instância de índice k recebe o dispositivo $i \cdot 2^{16} + k$. Assim, duas classes com a mesma unidade no mesmo nó nunca compartilham um dispositivo, que é o papel do parâmetro `dev` no conceito original.

Essa escolha tem duas propriedades favoráveis: primeiro, mantém as mensagens livres de qualquer campo de identificação adicional, pois o consumidor infere a mensagem e o campo de cada resposta a partir da unidade e do dispositivo, campos que já existem no SmartData; segundo, suporta sensores vetoriais com até 2^{16} elementos, suficiente para LiDARs comerciais de alta resolução. Como os interesses do *proxy* usam o dispositivo curinga, as respostas de outras classes com a mesma unidade também chegam a cada reconstrutor, que as descarta pela faixa antes de alterar a

amostra em construção.

4.7 MENSAGENS NA REDE

4.7.1 Formato das Mensagens

O conceito original deixa a codificação dos atributos a cargo do protocolo (Seção 2.5.5). Adotou-se o *layout* das mensagens TSTP da implementação SDAV (LISHA/UFSC, 2026) na escala de coordenadas de 32 bits em centímetros, de modo que o deslocamento de cada campo coincide com o daquela implementação. O Quadro 1 resume as quatro estruturas. Todos os campos estão na ordem de *bytes* do *host*, *little-endian* nas plataformas consideradas, sem *padding*, o valor é um número de ponto flutuante de 64 bits, conforme o tipo numérico declarado na unidade.

Quadro 1 – Mensagens SmartData na rede

Mensagem	**Tamanho**	**Conteúdo**
Cabeçalho comum	60 B	escala, tipo e versão; modo e operação; incerteza; confiança da localização; origem (x, y, z) em centímetros e t em microssegundos; unidade; dispositivo; assinatura; último salto
Interesse	104 B	cabeçalho; região (centro, raio, t_0, t_f); validade; período
Resposta	78 B	cabeçalho; validade; fragmento; valor. Sem o valor (70 B) nos anúncios e ocultações
Comando	92 B	cabeçalho, com o centro da região e t_0 na origem; raio; t_f; validade; período; valor

Fonte: elaborada pelo autor.

Os campos de assinatura são enviados zerados, pois a camada de segurança está fora do escopo, e o último salto repete a origem, pois a difusão ocorre em um único segmento de rede.

4.7.2 Transporte UDP Multicast

As mensagens são difundidas, uma por datagrama, em um grupo UDP multicast configurável, com *time-to-live* igual a um e *loopback* habilitado, de modo que produtores e consumidores podem compartilhar a mesma máquina ou o mesmo processo. A escolha de multicast permite que consumidores entrem e saiam da rede sem configuração explícita do produtor. O grupo multicast é, porém, apenas o meio de difusão, fazendo com que a seleção por unidade, dispositivo e região espaço-temporal é feita pelos próprios objetos SmartData ao processar interesses e respostas, e não pela inscrição no grupo.

4.7.3 Espelho em Tópico ROS

O caminho secundário republica cada atualização dos SmartData locais em um tópico ROS por tópico de origem, `/smartdata/<tópico>`, haja ou não interesse por ela na rede, usando a mensagem `SmartData` definida no próprio pacote. Esse espelho tem propósito exclusivamente instrumental, permitindo o uso das ferramentas padrão do ecossistema ROS para inspeção (`ros2 topic echo`), gravação em *rosbag* e visualização, sem que o consumo da informação semântica dependa dele. O espelho carrega a unidade, o valor, a origem, o dispositivo e a validade de cada amostra e pode ser desativado por parâmetro (`enable_ros_mirror:=false`) quando não houver necessidade.

4.8 O LADO CONSUMIDOR

O `proxy_node` instancia, para cada classe de transdutor de sensor declarada, um SmartData remoto com dispositivo curinga, na região e no período configurados por parâmetro, de modo que um único interesse cobre, por exemplo, todos os feixes de um LiDAR. As respostas recebidas são reagrupadas pela origem temporal, que o gateway preenche com o mesmo `header.stamp` para todos os campos de uma mensagem. Sensores escalares produzem a mensagem a cada resposta. Sensores compostos a produzem quando todos os campos da amostra chegaram, e sensores vetoriais, quando começa uma nova varredura. Como uma resposta perdida impediria que uma amostra composta se completasse e a última varredura só seria publicada com a chegada da seguinte, o *proxy* verifica também, periodicamente, as amostras em aberto. A cada período de liberação, configurado por `drain_period_ms`, uma amostra que não recebeu nenhuma resposta desde a verificação anterior é publicada com os campos que chegaram. Para os atuadores, o *proxy* mantém um SmartData remoto por instância, com o dispositivo específico do componente, e a escrita de cada campo de uma mensagem publicada em `<tópico>_command` produz o comando correspondente.

O *proxy* não se limita a recuperar o que o próprio gateway exporta, ele também recebe SmartData produzidos por outros nós da rede, como outro gateway ou um nó embarcado. Esses dados são declarados na seção `imports` do descritor, e cada entrada descreve o que pedir na rede e onde publicar o resultado. Do lado da rede, a entrada declara a unidade, por nome ou, quando o gerador não a conhece, pelo seu código de 32 bits, e o dispositivo do produtor, ou um curinga que aceita todos os produtores daquela unidade; região, período e validade podem ser definidos por entrada. Do lado do ROS, a entrada declara o tópico, o tipo da mensagem e o campo que recebe o valor. Como os dados vêm de fora, um import não tem transdutor local nem bloco de dispositivos, o *proxy* apenas anuncia o interesse e converte as respostas.

Dois aspectos exigem cuidado porque o produtor é desconhecido. O primeiro é

o tempo: a estampa da mensagem vem da origem da amostra, o que pressupõe que o produtor compartilhe a base de tempo do *Gateway*, condição discutida na Seção 4.1. O segundo é a identidade do produtor, com o dispositivo curinga, cada resposta gera uma mensagem, o `frame_id` pode identificar o dispositivo de origem, e as respostas dos transdutores do próprio pacote com a mesma unidade são ignoradas por padrão, para que o import receba apenas dados externos e não repita os que o próprio gateway exporta.

5 IMPLEMENTAÇÃO

Este capítulo descreve a implementação concreta do projeto descrito no capítulo anterior. A organização do código segue a estrutura de pacotes do ROS 2 `ament`, com o pacote `semantic_gateway` compreendendo as fontes em C++, o gerador em Python, o descritor de sensores em YAML, o arquivo de mensagem do espelho ROS e os arquivos de teste e de inicialização.

5.1 ESTRUTURA DO PACOTE

O Código 2 mostra a organização superficial do pacote ROS.

Código 2 – Estrutura de diretórios do pacote semantic_gateway

```
semantic_gateway/
  CMakeLists.txt
  package.xml
  msg/SmartData.msg
  config/sensors.yaml
  codegen/generate.py
  include/
    si_units.hpp
    smartdata/
      wire.hpp
      network.hpp
      transducer.hpp
      smartdata.hpp
      ros.hpp
  src/
    smartdata/network.cpp
    gateway_node.cpp
    proxy_node.cpp
    demo_publisher_node.cpp
  launch/
    gateway.launch.py
    gazebo_gateway.launch.py
  test/
    test_si_units.cpp
    test_smartdata_runtime.cpp
    test_gateway_pipeline.cpp
    test_bus_roundtrip.cpp
    test_imports.cpp
    imports_test.yaml
```

Fonte: elaborada pelo autor.

Os *headers* sob `build/generated/`, produzidos pelo gerador a cada compilação, complementam os *headers* estáticos sob `include/` no caminho de inclusão dos executáveis. O diretório `include/smartdata/`, junto com `src/smartdata/network.cpp`, forma o sistema de suporte à execução SmartData, compilado como uma biblioteca compartilhada usada pelos dois nós e pelos testes.

5.2 CODIFICAÇÃO DA UNIDADE SI

O cabeçalho `si_units.hpp` define a estrutura de *bit fields* que codifica a unidade segundo a Figura 1 de Fröhlich (2018). Cada uma das nove dimensões físicas recebe um campo de três bits com *offset* +4, permitindo representar expoentes inteiros de -4 a +3; dois bits identificam o tipo numérico (`Int32`, `Int64`, `Float32`, `Float64`); dois bits identificam um modificador opcional (direto, inverso, logarítmico); e um bit superior diferencia grandezas físicas de dados digitais. Constantes simbólicas como `METER_PER_SEC_SQ`, `KELVIN` e `VOLT` materializam essa codificação em tempo de compilação e são consumidas diretamente pelos *headers* gerados.

5.3 MENSAGENS E REDE

As estruturas `Header`, `Interest`, `Response` e `Command`, declaradas em `wire.hpp` com `#pragma pack(push,1)`, fixam o *layout* binário das mensagens do Quadro 1. Asserções estáticas sobre o tamanho de cada estrutura impedem que mudanças acidentais no *layout* comprometam silenciosamente a interoperabilidade, e um teste confere o deslocamento de cada campo. Os datagramas recebidos são copiados para as estruturas com `memcpy`, nunca por conversão de ponteiros, e um datagrama mais curto que a estrutura esperada é descartado. O mesmo arquivo define as representações usadas pelo restante do código, `Origin` e `Region`, em metros, e concentra em um único ponto a conversão entre metros e os centímetros inteiros da rede.

A classe `Network`, implementada em `network.cpp`, abre dois *sockets*: um de envio, com *time-to-live* e *loopback* configurados, e um de recepção, associado à porta e inscrito no grupo multicast. Uma *thread* receptora lê cada datagrama, interpreta apenas o seu cabeçalho e o entrega aos objetos SmartData registrados para a unidade correspondente, a interpretação do tipo da mensagem, do dispositivo e da região fica a cargo de cada objeto. A classe `Scheduler` executa, em uma única *thread*, as tarefas periódicas do sistema de suporte à execução, as respostas a interesses periódicos e a renovação dos interesses, e descarta ativações perdidas em vez de executá-las em rajada, pois, para um sensor, interessa o valor atual, e não os anteriores. Essa *thread* não participa do caminho dos interesses orientados a eventos, cujas respostas

são enviadas na própria *thread* que recebe a mensagem ROS. O seu trabalho por ativação se resume a montar e enviar um datagrama por associação periódica, e ela só se tornaria um gargalo se a taxa agregada dessas respostas se aproximasse da vazão de envio medida na Seção 6.4, da ordem de dezenas de milhares de datagramas por segundo. O relógio usado para as estampas de tempo é uma função passada ao construtor da `Network`.

5.4 OBJETOS SMARTDATA

O cabeçalho `smartdata.hpp` implementa os dois construtores do conceito original como dois modelos de classe. O `Responsive_SmartData<T>` possui o transdutor `T` e corresponde ao primeiro construtor: observa o transdutor, mantém as associações de interesse, responde e executa comandos. O `Interested_SmartData<UNIT>` corresponde ao segundo: anuncia e renova o interesse, filtra as respostas por unidade, dispositivo e região, aplica a função de fusão, quando houver, e envia comandos quando escrito. Ambos oferecem o operador de conversão para o valor, a validade e a espera bloqueante pela próxima atualização. O cabeçalho `transducer.hpp` define a base dos transdutores, que guarda a última leitura e notifica os seus observadores a cada amostra.

O cabeçalho `ros.hpp` faz a ligação com o ROS. O modelo `Source<Msg>` distribui cada mensagem de um tópico aos transdutores ligados a ele, e o modelo `Sink<Msg>` mantém o *setpoint* corrente de um atuador, aplica a escrita de cada componente e publica a mensagem, imediatamente ou ao fim da janela de coalescência, executada por uma *thread* própria. Como um mesmo SmartData é acessado pela *thread* do executor ROS, que entrega as mensagens, pela *thread* receptora da rede, que entrega interesses e comandos, e pela *thread* do escalonador, cada objeto protege o seu estado com uma exclusão mútua e invoca observadores e envios fora dela.

5.5 CAMINHO ROS PARA SMARTDATA

O `gateway_node` encapsula o sentido direto. Durante a construção, lê os parâmetros (`target_frame`, `enable_ros_mirror`, grupo e porta da rede), cria o *buffer* `tf2`, instancia a `Network` com o relógio do nó ROS e constrói o `Gateway` gerado, passando a função de localização e, se o espelho estiver habilitado, a função que publica cada atualização no tópico `/smartdata/<tópico>`. Cada *callback* de assinatura resolve a origem da mensagem na árvore `tf2`, com a confiança descrita na Seção 4.2, e a entrega ao `Source` do tópico, de onde ela segue aos transdutores e aos SmartData. Na destruição, o `Gateway` é destruído antes da `Network`, para que cada SmartData ainda possa anunciar a sua ocultação na rede.

5.6 O NÓ *PROXY*

O `proxy_node` é o nó simétrico ao `gateway_node`. Seus parâmetros definem a região de interesse (`region_x`, `region_y`, `region_z` e `region_radius`, sendo todo o espaço o padrão), o período dos interesses (`period_ms`, zero para interesses orientados a eventos) e o período de liberação das amostras vetoriais e compostas incompletas (`drain_period_ms`). Durante a construção, o nó instancia a `Network`, também com o relógio do ROS, e a classe `Proxies` gerada, que cria os publicadores `<tópico>_recovered`, os SmartData remotos e as assinaturas `<tópico>_command`. As respostas chegam pela *thread* receptora da rede, que atualiza o estado de reconstrução e publica a mensagem completa, um *timer* periódico, de 50 ms por padrão, libera as amostras que pararam de receber dados. Na destruição, os *proxies* revogam os seus interesses antes do fechamento da rede.

5.7 GERAÇÃO DE CÓDIGO

O gerador `codegen/generate.py` é um *script* Python que consome o YAML via `PyYAML` e produz os três *headers* já mencionados. A produção dos textos C++ é feita com *f-strings*. As principais responsabilidades do gerador são:

1. Validar o descritor e construir um modelo intermediário de tópicos, classes de transdutor e imports; qualquer erro, como unidade desconhecida, tópico repetido ou dispositivo importado dentro de um bloco local, interrompe a compilação com uma mensagem que aponta a entrada do YAML.

2. Alocar o bloco de dispositivos de cada classe de transdutor, respeitando o esquema $i \cdot 2^{16}$ descrito na Seção 4.6.

3. Emitir as classes de transdutor, com acesso direto aos campos, aplicação do `value_transform` declarado e dos filtros dos sensores vetoriais, e a escrita do componente nos atuadores.

4. Emitir a classe `Gateway`, com as assinaturas, os publicadores e os SmartData de cada instância.

5. Emitir a classe `Proxies`, com os SmartData remotos, a transformação inversa embutida, a lógica de agrupamento pela origem temporal, o descarte prévio de dispositivos de outras classes, o redimensionamento de vetores quando o índice ultrapassa a capacidade atual e os imports.

6 PLANO E EXECUÇÃO DE EXPERIMENTOS

Este capítulo apresenta o plano experimental, os procedimentos de medição e os resultados obtidos. A avaliação segue a abordagem de Jain (1991): cada experimento testa uma hipótese refutável, formulada antes da execução, com fatores, métricas e critério de decisão fixados de antemão, e os resultados são acompanhados de uma medida da sua variabilidade. As hipóteses refinam, em termos mensuráveis, a hipótese geral da Seção 1.2, e cobrem as métricas exigidas pela proposta, latência de tradução e vazão sustentada, o custo do envio das respostas pela rede, a latência de ponta a ponta através da rede SmartData e o seguimento de comandos por um robô simulado.

Os limiares adotados derivam do ecossistema em que o gateway opera. Para a latência, o alvo é permanecer ao menos uma ordem de grandeza abaixo do piso de latência do transporte ROS 2. Trabalhos de caracterização mostram, no 95º percentil, valores entre um e três milissegundos para cargas pequenas em ROS 2 (KRONAUER et al., 2021; MARUYAMA et al., 2016); adota-se, portanto, 100 µs como limite para a latência de tradução, o que garante contribuição desprezível ao orçamento total, e 1 ms, o próprio piso, como limite para o caminho completo pela rede SmartData. Para a vazão, as metas derivam de sensores comerciais usados em robôs móveis. Para a tradução, a referência é um LiDAR 3D, o Velodyne VLP-16, que produz cerca de 300.000 pontos por segundo (VELODYNE LIDAR, 2019): o gateway deve traduzir mais de $3 \times 10^5\,\mathrm{s}^{-1}$ atualizações de SmartData. Para a tradução com envio, em que cada atualização se torna um datagrama, a referência é um conjunto de sensores de alta taxa: um LiDAR 2D Hokuyo UTM-30LX, com 1.081 medições por varredura a 40 Hz (HOKUYO AUTOMATIC CO., 2012), cerca de 43.000 medições por segundo, e uma IMU Bosch BMI088, com giroscópio a 2 kHz e acelerômetro a 1,6 kHz em três eixos cada (BOSCH SENSORTEC, 2018), cerca de 11.000 medições por segundo. O gateway deve, portanto, enviar mais de $5{,}4 \times 10^4\,\mathrm{s}^{-1}$ datagramas.

6.1 AMBIENTE EXPERIMENTAL

Todos os experimentos foram executados em um mesmo computador, um *notebook* com processador Intel Core i7-1165G7 (quatro núcleos, oito *threads*, 2,8 GHz), sob Ubuntu 24.04 nativo, com núcleo Linux 6.17 configurado com preempção dinâmica (`PREEMPT_DYNAMIC`) e o regulador de frequência em modo `performance`. O *software* executa em um contêiner Docker baseado em Ubuntu 22.04, com ROS 2 Humble e Fast DDS (`rmw_fastrtps_cpp`), compilado com `colcon` em modo `Release` (GCC 11.4, `-O3`, asserções desabilitadas). O contêiner compartilha a pilha de rede do *host*, de modo que os datagramas da rede SmartData seguem a rota padrão do grupo multicast,

que nesse computador é a interface Wi-Fi: cada datagrama é entregue aos receptores locais por cópia do próprio núcleo e transmitido pela placa sem fio. Nenhuma *thread* foi fixada a um núcleo, e o computador não foi usado durante as medições.

6.2 HIPÓTESES

O Quadro 2 resume as hipóteses, os fatores variados e os critérios de decisão.

Quadro 2 – Hipóteses, fatores e critérios de decisão dos experimentos

	Hipótese	**Fatores**	**Critério de decisão**
H1	A latência de tradução cresce linearmente com o número de campos da mensagem e fica abaixo de 100 µs para as mensagens do descritor.	`LaserScan` com 10, 100, 360, 1.000 e 4.000 feixes; `Imu`; `Temperature`; sem envio.	Ajuste linear do p50 com $R^2 > 0{,}95$; p99 < 100 µs até 360 feixes, na IMU e na temperatura.
H2	Existe uma taxa R^* abaixo da qual a latência é estável e não há perdas; acima dela, uma fila de profundidade 5, como a do `SensorDataQoS`, transborda.	Taxa ofertada em laço aberto, sem envio ($K = 0$) e com 100 respostas por varredura ($K = 100$).	R^* é a maior taxa com perda $< 1\%$; R^* supera as metas de vazão.
H3	O custo do envio é linear no número K de respostas e se deve ao `sendto` do sistema operacional, não ao gateway.	$K = 0$, 1, 10, 50 e 100, no gateway e em três bases de comparação sem o gateway.	Ajuste linear com $R^2 > 0{,}95$; custo por datagrama do gateway até $1{,}5\times$ o do `sendto` nas mesmas condições.
H4	O caminho publicador ROS $\rightarrow$ gateway $\rightarrow$ rede $\rightarrow$ *proxy* $\rightarrow$ assinante ROS tem p95 abaixo de 1 ms e acrescenta ao salto ROS 2 direto menos que um salto adicional.	Taxa de publicação de `Imu`: 100, 500, 1.000, 2.000 e 5.000 Hz.	p95 $<$ 1 ms; perda $< 1\%$; sobrecarga do p50 menor que o p50 do salto direto.
H5	O robô segue os *setpoints* enviados pela rede SmartData, com erro de regime pequeno e atraso de comando de poucos milissegundos.	Sequência de degraus de velocidade linear e angular no Gazebo.	$\lvert\text{erro}\rvert <$ max(0,01; 5% do comando).

Fonte: elaborada pelo autor.

6.3 PROCEDIMENTOS

6.3.1 Repetição e Estatísticas

Cada configuração é executada várias vezes, cada vez em um processo novo, para que a variação entre execuções, decorrente do posicionamento da memória, do escalonamento e do estado do processador, apareça nos resultados: 30 repeti-

ções nos experimentos H1 e H3, cinco nos experimentos H2 e H4, e três no H5, cujas execuções duram minutos. Em cada repetição, as latências são medidas com `std::chrono::steady_clock`, após um aquecimento inicial descartado, e resumidas pelo mínimo, pelos percentis 5, 50, 95, 99 e 99,9 e pelo máximo. Entre as repetições, relata-se a mediana de cada percentil, acompanhada de um intervalo de confiança de 95% obtido por *bootstrap* de percentis (EFRON; TIBSHIRANI, 1993): as repetições são reamostradas com reposição e o intervalo é formado pelos percentis 2,5 e 97,5 das medianas reamostradas. O método não supõe uma distribuição para as latências, apenas que as repetições são independentes entre si. O máximo, por ser um evento isolado, é relatado apenas como o maior valor medido, sem inferência estatística. Os valores apresentados nas tabelas seguem esse formato, com o intervalo entre colchetes quando relevante.

O número de amostras necessário depende do percentil: o p50 depende do centro da distribuição e se estabiliza com poucas centenas de amostras, enquanto o p99 é determinado apenas pelo 1% mais lento e exige muitas mais. Por isso, a suficiência das amostras é avaliada em cada hipótese sobre a estatística que decide o critério: considera-se que o número de amostras basta quando o intervalo de confiança dessa estatística não cruza o limiar de decisão. Os percentis de cauda de configurações com poucas amostras por repetição são relatados apenas de forma descritiva, sem participar das decisões.

6.3.2 Latência de Tradução (H1)

O cenário sintético entrega mensagens diretamente ao `Source` gerado do tópico, com todos os transdutores e SmartData correspondentes instanciados, e mede o intervalo da chamada, que compreende a extração de cada campo, a aplicação dos filtros, a conversão de unidade, o registro de cada amostra e a notificação de cada SmartData. Nenhum interesse está associado, de modo que nada é enviado: o objetivo é isolar o custo da tradução propriamente dita. São medidas 20.000 mensagens por repetição, número definido a partir de uma série única de 200.000 traduções de um `LaserScan` de 100 feixes, sobre cujos prefixos crescentes se determinou a partir de quantas iterações cada percentil permanece a $\pm$5% do valor final (Seção 6.4.1). As mensagens medidas são `LaserScan` de 10 a 4.000 feixes, que traçam a curva de latência por tamanho e localizam o ponto em que o limite é ultrapassado, e as mensagens `Imu` e `Temperature` do descritor.

6.3.3 Carga em Laço Aberto (H2)

Em um laço fechado, cada mensagem só é produzida após o término da anterior, e a medição ignora o tempo que as mensagens passariam aguardando em uma fila sob carga real. Por isso, a vazão é medida em laço aberto: uma *thread* produtora

gera mensagens em instantes agendados a uma taxa fixa, independentemente do atendimento, e as deposita em uma fila que mantém apenas as cinco mais recentes, como a política *keep last* do `SensorDataQoS`, uma *thread* consumidora retira e traduz as mensagens. A latência é medida desde o instante agendado de chegada, e não desde o início do atendimento, incluindo a espera na fila. Cada taxa é mantida por 2 s, com e sem o envio de 100 respostas por varredura, e registram-se a taxa servida, a fração de mensagens perdidas por transbordo da fila e a latência. A decisão depende apenas da perda, contada sobre todas as mensagens ofertadas, nas taxas mais baixas, com poucas centenas de mensagens por repetição, o p99 da latência é descritivo.

6.3.4 Custo do Envio (H3)

Para medir o envio, K SmartData remotos registram interesse, cada um em um feixe específico, de modo que cada varredura de 100 feixes produz K respostas, cada uma em seu próprio datagrama, como no TSTP. O intervalo medido passa a incluir a construção e o envio dessas respostas, em 2.000 iterações por repetição. Para separar o custo do gateway do custo do sistema operacional, o mesmo número de datagramas é enviado sem o gateway, em três bases de comparação: um *socket* UDP cru; um *socket* cru com um receptor local inscrito no grupo multicast, condição em que o núcleo entrega uma cópia de cada datagrama ao próprio computador, como ocorre quando o *proxy* executa na mesma máquina; e a classe de rede do sistema de suporte à execução (`sd::Network`), que também mantém um receptor no grupo. O custo por datagrama é a inclinação do p50 em função de K, estatística que se estabiliza com as 2.000 iterações de cada repetição; o p95 e o p99 são relatados apenas para caracterizar a cauda.

6.3.5 Latência de Ponta a Ponta (H4)

Neste experimento executam, como processos separados, um publicador de mensagens `Imu`, o `gateway_node`, o `proxy_node`, um publicador da transformação estática do sensor e um nó de sondagem. O publicador carimba cada mensagem com o relógio no instante da publicação. A sonda assina tanto o tópico original, um salto ROS 2 direto que serve de base de comparação, quanto o tópico reconstruído pelo *proxy* exclusivamente a partir da rede SmartData, e registra, para cada mensagem, a diferença entre o instante de recepção e o carimbo. Como todos os processos leem o mesmo relógio, essa diferença é a latência do caminho. A mensagem `Imu` é composta: o *proxy* a republica assim que seus seis campos chegam, sem esperar a mensagem seguinte. Cada taxa é mantida por 10 s, o que resulta em 1.000 a 50.000 mensagens por repetição, e o número de mensagens recebidas em cada caminho é comparado ao número de mensagens publicadas. Na taxa mais baixa, o p95 de cada repetição

apoia-se em 50 amostras, e a sua suficiência é avaliada pelo intervalo de confiança entre as repetições.

6.3.6 Seguimento de Comandos (H5)

O cenário Gazebo (KOENIG et al., 2004) usa o robô TurtleBot3 *burger* em um mundo vazio, sob o tempo simulado. Um nó publica em `/cmd_vel_command`, a 20 Hz, uma sequência de seis degraus de velocidade linear e angular com 8 s de duração cada, o *proxy* converte cada mensagem em comandos SmartData, e o gateway, ao recebê-los, publica o *setpoint* em `/cmd_vel`. O erro de regime de cada degrau é a diferença entre o comando e a média da velocidade medida pela odometria do robô na segunda metade do degrau. O atraso de comando é o intervalo, em tempo real, entre a primeira publicação de cada degrau e o aparecimento do novo valor em `/cmd_vel`.

6.4 RESULTADOS

6.4.1 Latência de Tradução (H1)

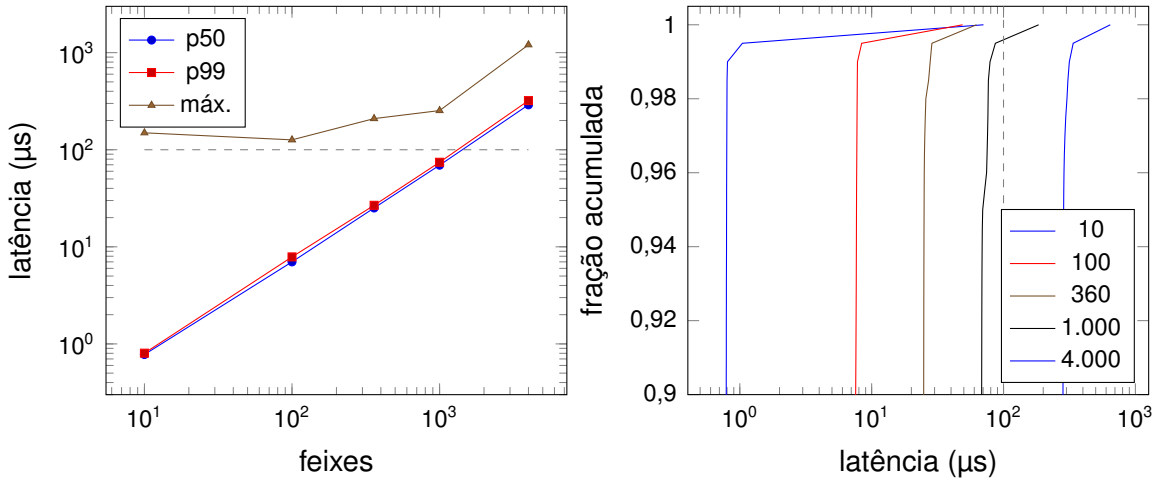
A Tabela 1 e a Figura 4 apresentam a latência de tradução por mensagem. O p50 do `LaserScan` cresce linearmente com o número de feixes, cerca de 72 ns por feixe, com $R^2 = 0{,}9987$, o que inclui, para cada feixe, a extração, o filtro, o registro da amostra e a notificação do SmartData. O p99 permanece abaixo de 100 µs até 1.000 feixes e ultrapassa o limite entre 1.000 e 4.000 feixes; pela inclinação do p99, o ponto de ruptura fica em torno de 1.300 feixes. Para as mensagens do descritor, o p99 é de 26,8 µs no `LaserScan` de 360 feixes, 0,51 µs na IMU e 0,10 µs na temperatura. A hipótese H1 é, portanto, confirmada.

Tabela 1 – Latência de tradução por mensagem (µs)

Mensagem	Campos	p50	p95	p99	p99,9	Máx.
`Temperature`	1	0,10	0,10	0,10	0,14	102,2
`Imu`	6	0,48	0,50	0,51	0,90	142,1
`LaserScan`	10	0,78	0,79	0,80	2,01	149,4
`LaserScan`	100	7,00	7,64	7,87	9,41	126,4
`LaserScan`	360	25,1	25,9	26,8	30,0	209,3
`LaserScan`	1.000	69,3	71,5	74,4	87,1	253,2
`LaserScan`	4.000	290,0	301,2	320,7	378,7	1.205,2

Fonte: elaborada pelo autor.
Notas: 1. Percentis: mediana de 30 repetições de 20.000 mensagens.
2. Máx.: maior valor medido entre as 600.000 traduções.

Figura 4 – Latência de tradução em função do número de feixes



Fonte: elaborada pelo autor.
Nota: À esquerda, percentis do `LaserScan`; à direita, cauda da distribuição acumulada de uma repetição, acima do 90º percentil. A linha tracejada marca o limite de 100 µs.

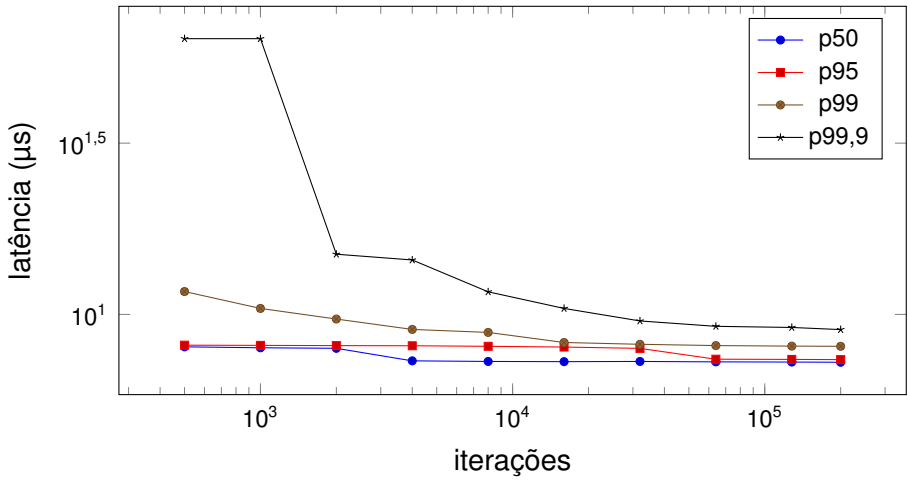
A suficiência das 20.000 iterações por repetição foi verificada sobre uma série única de 200.000 traduções. A Tabela 2 e a Figura 5 mostram os percentis em função do número de iterações. O p50 estabiliza a partir de 4.000 iterações, o p99 a partir de 16.000, e o p95 e o p99,9 a partir de 64.000. Com 500 iterações, o p99,9 superestima o valor final em sete vezes, e com 2.000 iterações o p99 ainda o superestima em 20%. A instabilidade do p95 tem outra origem: a distribuição é bimodal, com cerca de 85% das amostras perto de 6,7 µs e o restante perto de 7,6 µs (Figura 4), e o p95 fica na fronteira entre os dois modos, deslocando-se com a proporção entre eles. As 20.000 iterações usadas em cada repetição do H1 bastam, portanto, para o p50 e o p99, e a mediana de 30 repetições, com o seu intervalo de confiança estreito, compensa a sensibilidade do p95 e do p99,9.

Tabela 2 – Percentis da latência de tradução por número de iterações (µs)

Iterações	p50	p95	p99	p99,9
500	8,04	8,13	11,66	63,82
2.000	7,96	8,11	9,69	14,99
4.000	7,32	8,10	9,04	14,42
16.000	7,28	8,03	8,28	10,41
64.000	7,27	7,40	8,11	9,23
200.000	7,25	7,38	8,07	9,03

Fonte: elaborada pelo autor.
Nota: Prefixos crescentes de uma série de 200.000 traduções de um `LaserScan` de 100 feixes.

Figura 5 – Percentis da latência de tradução por número de iterações



Fonte: elaborada pelo autor.

6.4.2 Carga em Laço Aberto (H2)

A Tabela 3 e a Figura 6 mostram a taxa servida, as perdas e a latência em função da taxa ofertada. Sem envio, a latência mediana permanece em torno de 8,5 µs e não há perdas até 90.000 varreduras por segundo. A 120.000 varreduras por segundo, a perda chega a 1,1%, e a taxa servida satura em cerca de 118.700 varreduras por segundo, o inverso do tempo de serviço de 8,4 µs. Acima desse ponto, a fila permanece cheia, e a latência mediana passa a cerca de 40 µs, o tempo de atendimento das cinco mensagens à frente na fila, enquanto o excedente é descartado. Com 100 respostas por varredura, não há perdas até 1.000 varreduras, ou 100.000 datagramas, por segundo, e a taxa servida satura em cerca de 1.080 varreduras por segundo; a 1.000 varreduras por segundo, o p99 já sobe para 2,1 ms, sinal da proximidade da saturação.

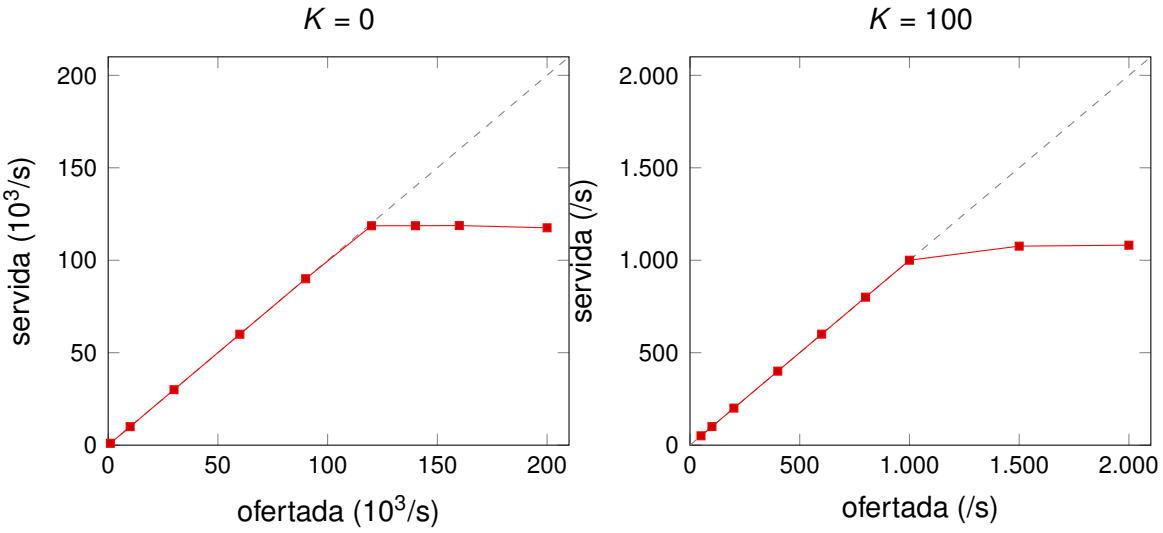
Tabela 3 – Carga em laço aberto

Sem envio (K = 0)					**Com envio (K = 100)**				
Ofertada (s^{-1})	**Servida** (s^{-1})	**Perda**	**p50** (µs)	**p99** (µs)	**Ofertada** (s^{-1})	**Servida** (s^{-1})	**Perda**	**p50** (µs)	**p99** (µs)
1.000	1.000	0%	7,5	8,8	50	50	0%	379	520
10.000	10.000	0%	8,5	9,4	100	100	0%	376	450
30.000	30.000	0%	8,5	11,2	200	200	0%	374	459
60.000	60.000	0%	8,5	11,9	400	400	0%	400	488
90.000	90.000	0%	8,6	18,1	600	600	0%	401	503
120.000	118.650	1,1%	40,1	50,0	800	800	0%	420	539
140.000	118.658	15%	40,6	44,3	1.000	1.000	0%	437	2.080
160.000	118.757	26%	36,5	39,8	1.500	1.076	28%	3.764	5.243
200.000	117.547	41%	31,0	33,8	2.000	1.081	46%	3.187	4.562

Fonte: elaborada pelo autor.
Nota: `LaserScan` de 100 feixes; taxa servida, perda por transbordo da fila e latência desde a chegada agendada, mediana de cinco repetições.

Assim, R^* é de 90.000 varreduras por segundo sem envio, entre as taxas avaliadas, o que corresponde a 9×10^6 atualizações de SmartData por segundo, trinta vezes a meta de $3 \times 10^5\,s^{-1}$ do LiDAR 3D, e de 1.000 varreduras por segundo com envio, ou 100.000 datagramas por segundo, quase o dobro da meta de $5{,}4 \times 10^4\,s^{-1}$ do LiDAR 2D com a IMU. A hipótese H2 é confirmada: o ponto de saturação existe, é delimitado, e fica acima das duas metas de vazão.

Figura 6 – Taxa servida em função da taxa ofertada



Fonte: elaborada pelo autor.
Nota: A linha tracejada indica a taxa servida igual à ofertada.

6.4.3 Custo do Envio (H3)

A Tabela 4 e a Figura 7 comparam o gateway às bases de comparação. Em todas as configurações com receptor local, o custo é praticamente o mesmo: com $K = 100$, uma varredura com tradução e envio custa 1.012 µs no gateway e 999 µs a 1.000 µs no *socket* com receptor e na classe de rede, diferença da ordem da própria tradução dos 100 feixes (7,6 µs). A inclinação do p50 em K é de 9,4 µs por datagrama no gateway e de 9,9 µs nas bases de comparação, razão de 1,0×. Com uma única resposta, o envio acrescenta 5,0 µs à tradução, contra 3,5 µs do `sendto` com receptor local, razão de 1,4×, ainda dentro do critério. A entrega local custa cerca de 1,3 µs por datagrama, diferença entre o *socket* cru com e sem receptor para $K = 1$.

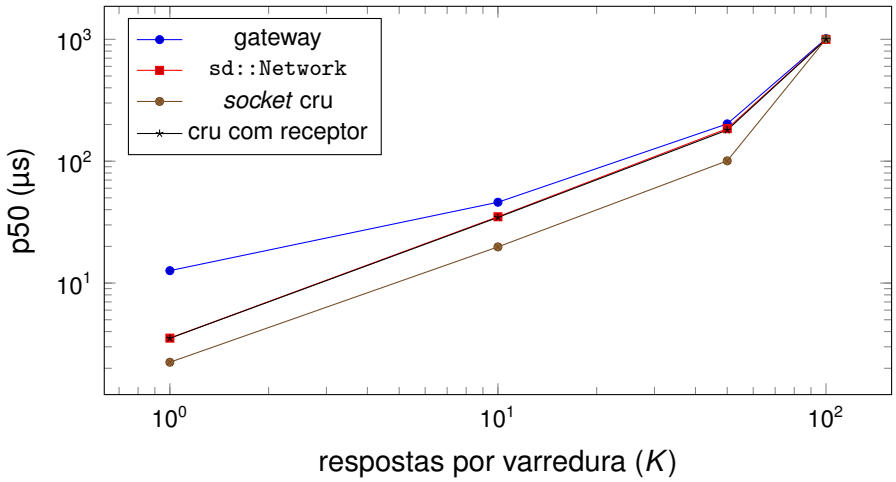
Tabela 4 – Latência por varredura em função do número de respostas (µs)

K	**Gateway**	`sd::Network`	***Socket* cru**	**Cru com receptor**
0	7,6 / 7,8	—	—	—
1	12,6 / 14,6	3,5 / 4,3	2,2 / 2,8	3,5 / 4,2
10	46 / 649	35 / 691	20 / 817	35 / 700
50	202 / 1.180	185 / 1.187	101 / 1.160	180 / 1.179
100	1.012 / 2.004	999 / 1.965	1.004 / 1.761	1.000 / 1.931

Fonte: elaborada pelo autor.
Nota: `LaserScan` de 100 feixes; p50 / p95, mediana de 30 repetições.

O custo, no entanto, não é linear em K. Por datagrama, o *socket* cru custa cerca de 2 µs até $K = 50$ e 10 µs com $K = 100$, e, a partir de $K = 10$, o p95 salta para centenas de microssegundos em todas as configurações. O ajuste linear resulta em $R^2 \approx 0{,}90$, abaixo do critério, e essa parte da hipótese é refutada. Como a não linearidade aparece de forma idêntica no *socket* cru, sem o gateway e sem receptor, ela não se deve ao gateway, mas ao caminho de saída do sistema operacional: o comportamento é compatível com o esgotamento do *buffer* de envio do *socket*, cujo espaço só é liberado quando os datagramas deixam a interface. Com poucas respostas, o `sendto` apenas enfileira o datagrama e retorna; com rajadas maiores, passa a aguardar a transmissão pela placa Wi-Fi, e o custo por datagrama converge para o da interface. A segunda parte da hipótese, que atribui o custo ao sistema operacional e não ao gateway, é confirmada.

Figura 7 – Latência por varredura em função do número de respostas



Fonte: elaborada pelo autor.
Nota: Mediana de 30 repetições, no gateway e nas bases de comparação sem o gateway.

6.4.4 Latência de Ponta a Ponta (H4)

A Tabela 5 e a Figura 8 comparam o salto ROS 2 direto ao caminho completo pela rede SmartData. Em todas as taxas, o p95 do caminho pelo gateway permaneceu abaixo de 1 ms, com o maior valor, 675 µs, a 100 Hz, com intervalo de confiança de 632 µs a 739 µs, distante do limiar, e a perda mediana foi nula; a maior perda em uma repetição isolada foi de 1,4%, a 5.000 Hz. A sobrecarga do p50 foi de 263 µs a 100 Hz, 91 µs a 500 Hz e entre 56 µs e 79 µs de 1.000 a 5.000 Hz. Ela é menor que o salto direto a 100 e 500 Hz, mas o supera de 1.000 a 5.000 Hz. As duas primeiras partes da hipótese H4 são confirmadas, e a terceira é refutada a partir de 1.000 Hz.

Tabela 5 – Latência de ponta a ponta (µs)

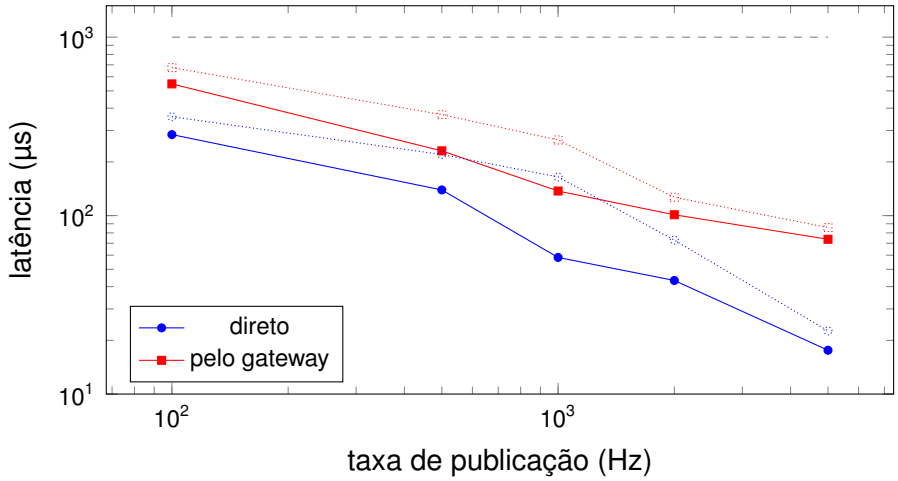
	Direto			Pelo gateway			
Taxa (Hz)	**p50**	**p95**	**p99**	**p50**	**p95**	**p99**	**Sobrecarga do p50**
100	284	358	384	547	675	731	+263
500	139	220	264	231	368	422	+91
1.000	58	165	198	137	265	303	+79
2.000	43	73	86	101	127	147	+58
5.000	18	23	27	74	86	100	+56

Fonte: elaborada pelo autor.
Nota: Mensagens `Imu` pelo salto ROS 2 direto e pelo caminho gateway → rede SmartData → *proxy*, mediana de cinco repetições.

O resultado mais saliente, porém, é a dependência da latência em relação à taxa: o próprio salto direto leva 284 µs a 100 Hz e 18 µs a 5.000 Hz, e a sobrecarga

do gateway acompanha essa tendência. Esse comportamento é compatível com os estados de economia de energia do processador: em taxas baixas, os processos dormem entre uma mensagem e outra, o núcleo entra em estados de repouso profundos, e cada mensagem paga o tempo de despertar do núcleo e do processo. O regulador de frequência em modo `performance` não desabilita esses estados. O caminho pelo gateway contém dois processos a mais, o gateway e o *proxy*, e paga esse custo em cada um deles. A 5.000 Hz, quando os processos quase não dormem, a sobrecarga de 56 µs aproxima o custo intrínseco do caminho: a tradução dos seis campos, inferior a um microssegundo, o envio de seis datagramas, a reconstrução da mensagem pelo *proxy* e um salto ROS 2 adicional, da republicação até a sonda. O critério relativo ao salto direto mostrou-se, assim, inadequado para taxas altas, em que o salto direto, sem tradução nem rede, é excepcionalmente barato; o critério absoluto, de p95 abaixo do piso de 1 ms, foi atendido em todas as taxas.

Figura 8 – Latência de ponta a ponta em função da taxa de publicação



Fonte: elaborada pelo autor.
Nota: Marcadores cheios: mediana; vazios: 95º percentil. A linha tracejada marca o limite de 1 ms.

6.4.5 Seguimento de Comandos (H5)

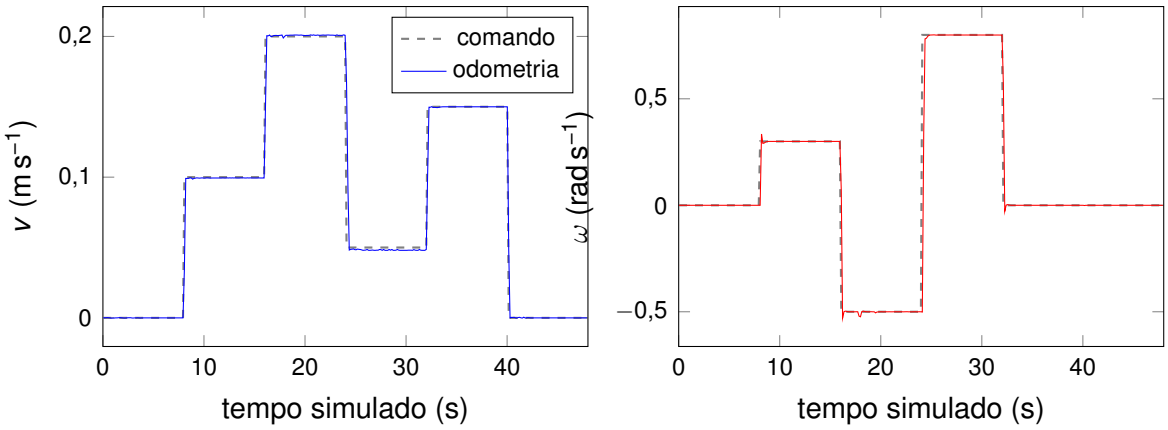
A Tabela 6 e a Figura 9 mostram a resposta do robô aos degraus de velocidade. O erro de regime foi o mesmo nas três repetições, com o maior valor, $1{,}8 \times 10^{-3}\,\mathrm{m\,s^{-1}}$, no degrau de $0{,}05\,\mathrm{m\,s^{-1}}$ e $0{,}80\,\mathrm{rad\,s^{-1}}$, 3,6% do comando e abaixo da tolerância de 0,01. O atraso de comando, entre a publicação em `/cmd_vel_command` e o aparecimento do *setpoint* em `/cmd_vel`, ficou entre 2,15 ms e 2,41 ms, com mediana de 2,34 ms, dos quais 2 ms correspondem à janela de coalescência do atuador, que agrupa os campos de uma mesma mensagem em um único *setpoint* (Seção 4.5); o restante, cerca de 0,3 ms, corresponde aos dois saltos ROS 2 e ao envio pela rede. A hipótese H5 é confirmada.

Tabela 6 – Erro de regime e atraso de comando por degrau

Degrau	v (m s^{-1})	ω (rad s^{-1})	**Erro de** v	**Erro de** ω	**Atraso** (ms)
0	0,00	0,00	$< 10^{-4}$	$< 10^{-4}$	2,33–2,38
1	0,10	0,30	-6×10^{-4}	$< 10^{-4}$	2,25–2,36
2	0,20	−0,50	$+9 \times 10^{-4}$	-3×10^{-4}	2,20–2,34
3	0,05	0,80	$-1{,}8 \times 10^{-3}$	-5×10^{-4}	2,15–2,41
4	0,15	0,00	$< 10^{-4}$	$< 10^{-4}$	2,17–2,38
5	0,00	0,00	$< 10^{-4}$	$< 10^{-4}$	2,28–2,39

Fonte: elaborada pelo autor.
Nota: Três repetições do cenário Gazebo.

Figura 9 – Velocidades comandadas e medidas no Gazebo



Fonte: elaborada pelo autor.
Nota: Velocidade linear à esquerda e angular à direita, comandadas pela rede Smart-Data e medidas pela odometria do robô, na primeira repetição.

6.4.6 Síntese

O Quadro 3 reúne o veredito de cada hipótese. As hipóteses H1, H2 e H5 foram confirmadas. Das hipóteses H3 e H4, foram confirmadas as partes que tratam do custo do gateway e do limite absoluto de latência, e refutadas as que supunham linearidade do envio e uma sobrecarga menor que um salto ROS 2 em todas as taxas; em ambos os casos, a causa identificada está na plataforma, e não no gateway.

Quadro 3 – Síntese dos resultados frente aos critérios de decisão

	Critério	**Medido**	**Veredito**
H1	Linear, $R^2 > 0{,}95$; p99 $< 100\,\mu s$ até 360 feixes	72 ns/feixe, $R^2 = 0{,}999$; p99 de 26,8 µs (360 feixes); ruptura em cerca de 1.300 feixes	Confirmada
H2	R^* com perda $< 1\%$ acima de 3×10^5 atualizações/s sem envio e de $5{,}4 \times 10^4$ datagramas/s com envio	9×10^6 atualizações/s sem envio; 10^5 datagramas/s com envio	Confirmada
H3	Linear, $R^2 > 0{,}95$; gateway até $1{,}5\times$ o `sendto`	$R^2 \approx 0{,}90$; $1{,}0\times$ (inclinação) a $1{,}4\times$ ($K = 1$)	Linearidade refutada; atribuição confirmada
H4	p95 < 1 ms; perda $< 1\%$; sobrecarga $<$ salto direto	p95 $\leq 675\,\mu s$; perda mediana nula; sobrecarga de +56 a +263 µs	Limite absoluto confirmado; sobrecarga relativa refutada a partir de 1.000 Hz
H5	$\lvert\text{erro}\rvert < \max(0{,}01;\ 5\%)$	Erro máximo de $1{,}8 \times 10^{-3}$; atraso de 2,15 ms a 2,41 ms	Confirmada

Fonte: elaborada pelo autor.

6.5 DISCUSSÃO

Os resultados sustentam a hipótese geral declarada na Seção 1.2. A tradução custa 72 ns por campo e, para as mensagens do descritor, fica mais de três vezes abaixo do limite de 100 µs no p99, e o caminho completo pela rede SmartData fica abaixo do piso de latência do ROS 2 em todas as taxas avaliadas, preservando as mensagens e os comandos. As limitações observadas merecem alguns comentários.

Primeiro, o envio domina o custo total. Com um consumidor interessado em todos os feixes de uma varredura de 100 feixes, o envio custa cerca de 1 ms, mais de cem vezes a tradução. Esse custo decorre de cada SmartData ser uma mensagem própria, como no TSTP, e é limitado pela interface de rede, e não pelo gateway: no computador avaliado, a interface Wi-Fi impõe um teto de cerca de 100.000 datagramas por segundo, e uma interface cabeada ou dedicada deslocaria esse teto. Esse teto basta para um LiDAR 2D de alta resolução com uma IMU de alta taxa, mas não para um LiDAR 3D, cujos 300.000 pontos por segundo, se todos fossem requisitados por um consumidor, excederiam em três vezes a capacidade de envio. O custo só ocorre quando há interesse pelos dados, pois um SmartData sem interesse não envia nada. O agrupamento de respostas em um mesmo datagrama reduziria o número de chamadas de sistema e de quadros transmitidos, ao custo de se afastar do formato de referência.

Segundo, as latências dependem da taxa das mensagens. Em taxas baixas, típicas de sensores como a IMU a 100 Hz, os estados de economia de energia do

processador acrescentam centenas de microssegundos a cada salto, tanto no ROS 2 direto quanto no caminho pelo gateway. Em uma aplicação com requisitos de tempo real, esses estados podem ser limitados pelo sistema operacional, configuração não avaliada neste trabalho. Do mesmo modo, o máximo das latências é determinado pelas interrupções e preempções da plataforma, e só um núcleo com preempção total e núcleos isolados poderia oferecer garantias sobre o pior caso.

Terceiro, os experimentos foram executados em um único computador, um *notebook* de uso geral, com todos os processos na mesma máquina, de modo que a rede SmartData foi exercitada pela entrega local e pela saída da interface Wi-Fi, sem um receptor remoto. Resultados em *hardware* embarcado, típico em robôs de campo, não foram avaliados. Nesses processadores, a menor frequência, a menor capacidade de cache e a gestão de energia devem elevar a latência de tradução, que depende quase exclusivamente do desempenho de um único núcleo; o critério de aceitação seguiria cumprido, para o `LaserScan` de 360 feixes, em um processador até cerca de três vezes e meia mais lento que o utilizado, hipótese a ser verificada experimentalmente.

Quarto, a implementação se afasta do conceito original em três pontos. Sem função de fusão, todo SmartData que satisfaz um interesse responde, pois a supressão de respostas de outros nós não foi implementada; com períodos distintos sobre o mesmo transdutor, todos os interesses periódicos são respondidos no máximo divisor comum, e não cada um no seu período; e os interesses são renovados periodicamente, o que acrescenta um datagrama por segundo por interesse, mas torna a associação robusta à perda de pacotes e à ordem de inicialização dos nós.

7 CONSIDERAÇÕES FINAIS

Este trabalho apresentou o projeto, a implementação e a avaliação de um *Gateway* semântico que exporta sensores e atuadores ROS 2 como transdutores SmartData. A solução foi conduzida em três frentes. No plano conceitual, especificou-se, por meio de um descritor declarativo em YAML, o mapeamento entre os campos das mensagens robóticas e as classes de transdutor do SmartData, identificando três padrões de decomposição suficientes para cobrir os tipos relevantes da biblioteca `sensor_msgs`. No plano arquitetural, adotou-se a geração de código prévia à compilação como caminho único de tradução e um sistema de suporte à execução próprio que reproduz o modelo de execução de transdutores do conceito original, com modos de operação, interesses orientados a eventos e periódicos, regiões espaço-temporais, validade e comandos, em mensagens com o formato do TSTP. No plano da validação, cinco hipóteses refutáveis foram testadas em Linux nativo, com repetições e intervalos de confiança. Demonstrou-se que a latência de tradução cresce linearmente com o tamanho da mensagem, cerca de 72 ns por campo, e permanece abaixo do critério de aceitação derivado do piso de latência do ROS 2 (KRONAUER et al., 2021) para as mensagens do descritor; que o custo adicional do envio é determinado pelo sistema operacional e pela interface de rede, e não pelo gateway; que o caminho completo pela rede SmartData fica abaixo de 1 ms no 95º percentil; e que um robô simulado segue os comandos recebidos pela rede SmartData com erro de regime desprezível.

A contribuição complementa as implementações existentes do conceito de SmartData, predominantemente voltadas para nós de sensores embarcados, oferecendo uma ponte, nos dois sentidos, entre o ecossistema robótico e plataformas *IoT*, sem demandar reescrita dos sensores existentes ou intermediação por *brokers* de aplicação.

7.1 TRABALHOS FUTUROS

A continuidade natural deste trabalho organiza-se em cinco direções. Primeiro, a verificação experimental contra uma implementação de SmartData desenvolvida de forma independente, como a SDAV ou um nó EPOS sobre TSTP, comunicando-se com o *Gateway* pela mesma rede, fecharia o ciclo de interoperabilidade prometido pelo formato compartilhado. Segundo, a extensão do mecanismo de autenticação das mensagens, hoje preenchido com zeros, completaria as garantias de autenticidade e integridade dos dados originalmente previstas por Fröhlich (2018). Terceiro, a implementação da supressão de respostas na ausência de função de fusão e do atendimento de cada interesse periódico no seu próprio período aproximaria o sistema de suporte à execução do conceito original. Quarto, a avaliação do agrupamento de respostas por

datagrama permitiria reduzir o custo de envio dominante. Quinto, a caracterização de desempenho em *hardware* embarcado representativo do que se encontra em robôs de campo permitiria contrastar os resultados deste trabalho com cenários sob restrições mais severas de processador, memória e banda, e a repetição das medições com um núcleo de tempo real, núcleos isolados e estados de repouso limitados permitiria caracterizar o pior caso, hoje determinado pela plataforma.

REFERÊNCIAS

BOSCH SENSORTEC. *BMI088: 6-axis Motion Tracking for High-performance Applications. Data Sheet BST-BMI088-DS001*. 2018. Folha de dados. Disponível em: <https://www.bosch-sensortec.com/products/motion-sensors/imus/bmi088/>. Acesso em: 8 nov. 2026.

EFRON, B.; TIBSHIRANI, R. J. *An Introduction to the Bootstrap*. New York: Chapman & Hall, 1993.

EUGSTER, P. T. et al. The many faces of publish/subscribe. *ACM Computing Surveys*, v. 35, n. 2, p. 114–131, 2003.

FRÖHLICH, A. A. SmartData: an IoT-ready API for sensor networks. *International Journal of Sensor Networks*, v. 28, n. 3, p. 202–210, 2018.

FRÖHLICH, A. A. et al. Data-centric cyber-physical systems design with SmartData. In: *Proceedings of the 2018 Winter Simulation Conference (WSC)*. Göteborg, Sweden: IEEE, 2018. p. 1274–1285.

HOKUYO AUTOMATIC CO. *Scanning Laser Range Finder UTM-30LX: Specification*. 2012. Folha de dados. Disponível em: <https://www.hokuyo-aut.jp/search/single.php?serial=169>. Acesso em: 8 nov. 2026.

JAIN, R. *The Art of Computer Systems Performance Analysis: Techniques for Experimental Design, Measurement, Simulation, and Modeling*. New York: Wiley, 1991.

KOENIG, N. et al. Design and use paradigms for Gazebo, an open-source multi-robot simulator. In: *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. Sendai, Japan: IEEE, 2004. p. 2149–2154.

KOPETZ, H. *Real-Time Systems: Design Principles for Distributed Embedded Applications*. 2. ed. New York: Springer, 2011.

KRONAUER, T. et al. Latency analysis of ROS2 multi-node systems. In: *IEEE International Conference on Multisensor Fusion and Integration for Intelligent Systems (MFI)*. Karlsruhe, Germany: IEEE, 2021. p. 1–7.

LEE, E. A. Cyber physical systems: Design challenges. In: *11th IEEE International Symposium on Object and Component-Oriented Real-Time Distributed Computing (ISORC)*. Orlando, FL, USA: IEEE, 2008. p. 363–369.

LISHA/UFSC. *SDAV: implementação SmartData do projeto de veículos autônomos do LISHA*. 2026. Repositório de código-fonte. Disponível em: <https://gitlab.lisha.ufsc.br/iot/sdav>. Acesso em: 20 out. 2026.

MACENSKI, S. et al. Robot operating system 2: Design, architecture, and uses in the wild. *Science Robotics*, v. 7, n. 66, p. eabm6074, 2022.

MARUYAMA, Y. et al. Exploring the performance of ROS2. In: *Proceedings of the 13th International Conference on Embedded Software (EMSOFT)*. Pittsburgh, PA, USA: ACM, 2016. p. 1–10.

PRESTES, E. et al. Towards a core ontology for robotics and automation. *Robotics and Autonomous Systems*, v. 61, n. 11, p. 1193–1204, 2013.

RESNER, D. et al. Design rationale of a cross-layer, trustful space-time protocol for wireless sensor networks. In: *IEEE International Conference on Emerging Technologies and Factory Automation (ETFA)*. Luxembourg: IEEE, 2015. p. 1–8.

SERRANO, M.; HAUSWIRTH, M.; SOLDATOS, J. Defining the stack for service delivery models and interoperability in the Internet of Things: A practical case with the OpenIoT-VDK. *IEEE Journal on Selected Areas in Communications*, v. 33, n. 4, p. 676–689, 2015.

VELODYNE LIDAR. *Puck VLP-16: Datasheet 63-9229 Rev H*. 2019. Folha de dados. Disponível em: <https://velodynelidar.com/products/puck/>. Acesso em: 8 nov. 2026.